



Red Hat OpenShift AI Self-Managed 3.5

Working with AI pipelines

Work with AI pipelines from Red Hat OpenShift AI Self-Managed

Red Hat OpenShift AI Self-Managed 3.5 Working with AI pipelines

Work with AI pipelines from Red Hat OpenShift AI Self-Managed

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

Enhance your projects on OpenShift AI by building portable machine learning (ML) workflows with AI pipelines.

Table of Contents

PREFACE	5
CHAPTER 1. MANAGE AI PIPELINES	6
1.1. CONFIGURE A PIPELINE SERVER	6
1.1.1. Configure a pipeline server with an external Amazon RDS database	8
1.2. OPTIONS FOR DEFINING A PIPELINE	9
1.2.1. Compile the pipeline YAML with the Kubeflow Pipelines SDK	10
1.2.2. Compile Kubernetes-native manifests with the Kubeflow Pipelines SDK	10
1.2.3. Authenticate the Kubeflow Pipelines SDK with a pipeline server	12
1.2.4. Define a pipeline by using the Kubernetes API	14
1.2.5. Migrate pipelines from database to Kubernetes API storage	17
1.3. IMPORT A PIPELINE TO THE DASHBOARD	20
1.4. DELETE A PIPELINE	21
1.5. DELETE A PIPELINE SERVER	21
1.6. VIEW PIPELINE SERVER DETAILS	22
1.7. VIEW EXISTING PIPELINES	22
1.8. PIPELINE VERSIONS	23
1.9. UPLOAD A PIPELINE VERSION	23
1.10. DELETE A PIPELINE VERSION	24
1.11. VIEW PIPELINE VERSION DETAILS	25
1.12. DOWNLOAD A PIPELINE VERSION	25
1.13. PIPELINE CACHING	26
1.13.1. Caching criteria	26
1.13.2. Viewing cached steps in the OpenShift AI user interface	26
1.13.3. Control caching in pipelines	27
1.13.3.1. Disabling caching for individual tasks	27
1.13.3.2. Disabling caching for a pipeline at submit time	27
1.13.3.3. Disabling caching for a pipeline at compile time	28
1.13.3.4. Disabling caching for all pipelines (pipeline server)	28
CHAPTER 2. MANAGE PIPELINE EXPERIMENTS	30
2.1. PIPELINE EXPERIMENTS FOR ORGANIZED RUN COMPARISON	30
2.2. CREATING A PIPELINE EXPERIMENT	30
2.3. ARCHIVING A PIPELINE EXPERIMENT	31
2.4. DELETING AN ARCHIVED PIPELINE EXPERIMENT	31
2.5. RESTORING AN ARCHIVED PIPELINE EXPERIMENT	32
2.6. VIEWING PIPELINE TASK EXECUTIONS	32
2.7. VIEWING PIPELINE ARTIFACTS	33
2.8. COMPARING RUNS IN AN EXPERIMENT	34
2.9. COMPARING RUNS IN DIFFERENT EXPERIMENTS	35
CHAPTER 3. MANAGE PIPELINE RUNS	37
3.1. PIPELINE RUN TYPES AND LIFECYCLE	37
3.2. DATA STORAGE WITH PIPELINES	38
3.3. PIPELINE RUN WORKSPACES	38
3.3.1. Configure default workspace PVC settings in DSPA	39
3.3.2. Add external artifacts to pipeline run workspaces	41
3.4. VIEW ACTIVE PIPELINE RUNS	42
3.5. EXECUTE A PIPELINE RUN	43
3.6. STOP AN ACTIVE PIPELINE RUN	44
3.7. DUPLICATE AN ACTIVE PIPELINE RUN	44
3.8. VIEW SCHEDULED PIPELINE RUNS	45

3.9. PIPELINE RUN SCHEDULING WITH CRON JOBS	46
3.10. SCHEDULE A PIPELINE RUN	47
3.11. DUPLICATE A SCHEDULED PIPELINE RUN	48
3.12. DELETE A SCHEDULED PIPELINE RUN	50
3.13. VIEW PIPELINE RUN DETAILS	51
3.14. VIEW ARCHIVED PIPELINE RUNS	51
3.15. ARCHIVE A PIPELINE RUN	52
3.16. RESTORE AN ARCHIVED PIPELINE RUN	53
3.17. DELETE AN ARCHIVED PIPELINE RUN	53
3.18. DUPLICATE AN ARCHIVED PIPELINE RUN	54
CHAPTER 4. TRACK PIPELINE EXPERIMENTS WITH MLFLOW	56
4.1. MLFLOW EXPERIMENT TRACKING FOR PIPELINE RUNS	56
4.1.1. MLflow run mapping structure	56
4.1.2. Automatically logged data	56
4.1.3. Default experiment behavior	56
4.1.4. Graceful degradation	57
4.1.5. Direct SDK access from pipeline tasks	57
4.1.6. MLflow run view compared to the pipeline run view	57
4.2. ENABLE MLFLOW INTEGRATION FOR PIPELINE RUNS	57
4.3. VIEW MLFLOW EXPERIMENTS FOR PIPELINE RUNS	59
CHAPTER 5. WORK WITH PIPELINE LOGS	61
5.1. PIPELINE LOGS	61
5.2. VIEW PIPELINE STEP LOGS	61
5.3. DOWNLOAD PIPELINE STEP LOGS	62
CHAPTER 6. USE THE AI PIPELINES REST API WITH KUBEFLOW PIPELINES SDK	64
6.1. AI PIPELINES REST API AND SDK	64
6.1.1. REST API overview	64
6.1.2. Kubeflow pipelines SDK	64
6.1.3. Additional resources	64
6.2. INSTALL THE KUBEFLOW PIPELINES SDK	64
6.3. WORK WITH PIPELINES BY USING THE KUBEFLOW PIPELINES SDK	65
6.3.1. Uploading a pipeline	65
6.3.2. Listing pipelines	66
6.3.3. Getting pipeline details	67
6.3.4. Deleting a pipeline	67
6.3.5. Additional resources	67
6.4. WORK WITH PIPELINE VERSIONS BY USING THE KUBEFLOW PIPELINES SDK	67
6.4.1. Creating pipeline versions	68
6.4.2. Listing pipeline versions	68
6.4.3. Getting pipeline version details	69
6.4.4. Additional resources	69
6.5. WORK WITH EXPERIMENTS BY USING THE KUBEFLOW PIPELINES SDK	69
6.5.1. Creating an experiment	69
6.5.2. Listing experiments	70
6.5.3. Getting experiment details	70
6.5.4. Additional resources	71
6.6. WORK WITH RUNS BY USING THE SDK	71
6.6.1. Creating and submitting runs	71
6.6.2. Monitoring runs	72
6.6.3. Listing runs	73
6.6.4. Managing run lifecycle	74

6.6.5. Additional resources	74
6.7. WORKFLOW EXAMPLES FOR THE KUBEFLOW PIPELINES SDK	74
6.7.1. End-to-end workflow example	75
6.7.2. Example: Pagination for large result sets	77
6.7.3. Additional resources	77
CHAPTER 7. WORK WITH PIPELINES IN JUPYTERLAB	78
7.1. JUPYTERLAB VISUAL PIPELINE EDITOR	78
7.2. ACCESS THE PIPELINE EDITOR	79
7.3. DISABLE NODE CACHING IN ELYRA	80
7.4. CREATE A RUNTIME CONFIGURATION	81
7.5. UPDATE A RUNTIME CONFIGURATION	84
7.6. DELETE A RUNTIME CONFIGURATION	86
7.7. DUPLICATE A RUNTIME CONFIGURATION	87
7.8. RUN A PIPELINE IN JUPYTERLAB	88
7.9. EXPORT A PIPELINE IN JUPYTERLAB	89
CHAPTER 8. DSPA COMPONENT TROUBLESHOOTING	91
8.1. COMMON ERRORS ACROSS DSPA COMPONENTS	93
CHAPTER 9. ADDITIONAL RESOURCES	95

PREFACE

As a data scientist, you can enhance your projects on OpenShift AI by building portable machine learning (ML) workflows with AI pipelines, using Docker containers. This enables you to standardize and automate machine learning workflows to enable you to develop and deploy your data science models.

For example, the steps in a machine learning workflow might include items such as data extraction, data processing, feature extraction, model training, model validation, and model serving. Automating these activities enables your organization to develop a continuous process of retraining and updating a model based on newly received data. This can help address challenges related to building an integrated machine learning deployment and continuously operating it in production.

You can also use the Elyra JupyterLab extension to create and run AI pipelines within JupyterLab.

To use an AI pipeline in OpenShift AI, you need the following components:

- **Pipeline server:** A server that is attached to your project and hosts your AI pipeline.
- **Pipeline:** A pipeline defines the configuration of your machine learning workflow and the relationship between each component in the workflow.
 - Pipeline code: A definition of your pipeline in a YAML file.
 - Pipeline graph: A graphical illustration of the steps executed in a pipeline run and the relationship between them.
- **Pipeline experiment:** A workspace where you can try different configurations of your pipelines. You can use experiments to organize your runs into logical groups.
 - Archived pipeline experiment: An archived pipeline experiment.
 - Pipeline artifact: An output artifact produced by a pipeline component.
 - Pipeline execution: The execution of a task in a pipeline.
- **Pipeline run:** An execution of your pipeline.
 - Active run: A pipeline run that is executing, or stopped.
 - Scheduled run: A pipeline run that is scheduled to run at least once.
 - Archived run: An archived pipeline run.

This feature is based on Kubeflow Pipelines 2.0. Use the latest Kubeflow Pipelines 2.0 SDK to build your pipeline in Python code. After you have built your pipeline, use the SDK to compile it into an Intermediate Representation (IR) YAML file. The OpenShift AI user interface enables you to track and manage pipelines, experiments, and pipeline runs. To view a record of previously executed, scheduled, and archived runs, you can go to **Develop & train > Pipelines → Runs**, or you can select an experiment from the **Develop & train → Experiments** page to access all of its pipeline runs. You can manage incremental changes to pipelines in OpenShift AI by using versioning. This allows you to develop and deploy pipelines iteratively, preserving a record of your changes.

You can store your pipeline artifacts in an S3-compatible object storage bucket so that you do not consume local storage. To do this, you must first configure write access to your S3 bucket on your storage account.

CHAPTER 1. MANAGE AI PIPELINES

1.1. CONFIGURE A PIPELINE SERVER

Before you can successfully create an AI pipeline in OpenShift AI, you must configure a pipeline server. This task includes configuring where your pipeline artifacts and data are stored.



NOTE

You are not required to specify any storage directories when configuring a connection for your pipeline server. When you import a pipeline, the **/pipelines** folder is created in the **root** folder of the bucket, containing a YAML file for the pipeline. If you upload a new version of the same pipeline, a new YAML file with a different ID is added to the **/pipelines** folder.

When you run a pipeline, the artifacts are stored in the **/pipeline-name** folder in the **root** folder of the bucket.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project that you can add a pipeline server to.
- You have an existing S3-compatible object storage bucket and you have configured write access to your S3 bucket on your storage account.
- If you are configuring a pipeline server for production pipeline workloads, you have an existing external MySQL or MariaDB database.
- If you are configuring a pipeline server with an external MySQL database, your database must use at least MySQL version 5.x. However, Red Hat recommends that you use MySQL version 8.x.



NOTE

The **mysql_native_password** authentication plugin is required for the ML Metadata component to successfully connect to your database.

mysql_native_password is disabled by default in MySQL 8.4 and later. If your database uses MySQL 8.4 or later, you must update your MySQL deployment to enable the **mysql_native_password** plugin.

For more information about enabling the **mysql_native_password** plugin, see [Native Pluggable Authentication](#) in the MySQL documentation.

- If you are configuring a pipeline server with a MariaDB database, your database must use MariaDB version 10.3 or later. However, Red Hat recommends that you use at least MariaDB version 10.5.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.

2. On the **Projects** page, click the name of the project that you want to configure a pipeline server for.
The project details page opens.
3. Click the **Pipelines** tab.
4. Click **Configure pipeline server**.
The **Configure pipeline server** dialog opens.
5. In the **Object storage connection** section, provide values for the mandatory fields:
 - a. In the **Access key** field, enter the access key ID for the S3-compatible object storage provider.
 - b. In the **Secret key** field, enter the secret access key for the S3-compatible object storage account that you specified.
 - c. In the **Endpoint** field, enter the endpoint of your S3-compatible object storage bucket.
 - d. In the **Region** field, enter the default region of your S3-compatible object storage account.
 - e. In the **Bucket** field, enter the name of your S3-compatible object storage bucket.



IMPORTANT

If you specify incorrect connection settings, you cannot update these settings on the same pipeline server. Therefore, you must delete the pipeline server and configure another one.

If you want to use an existing artifact that was not generated by a task in a pipeline, you can use the [kfp.dsl.importer component](#) to import the artifact from its URI. You can only import these artifacts to the S3-compatible object storage bucket that you define in the **Bucket** field in your pipeline server configuration. For more information about the **kfp.dsl.importer** component, see [Special Case: Importer Components](#).

6. Click **Advanced settings** to display the **Database**, **Pipeline definition storage**, and **Pipeline caching** sections.
7. In the **Database** section, choose one of the following options to specify where to store your pipeline metadata and run information:
 - Select **Default database on the cluster** to deploy a MariaDB database in your project.



IMPORTANT

The **Default database on the cluster** option is intended for development and testing purposes only. For production pipeline workloads, select the **External MySQL database** option to use an external MySQL or MariaDB database.

- Select **External MySQL database** to add a new connection to an external MySQL or MariaDB database that your pipeline server can access.
 - i. In the **Host** field, enter the database hostname.

- ii. In the **Port** field, enter the database port.
 - iii. In the **Username** field, enter the default user name that is connected to the database.
 - iv. In the **Password** field, enter the password for the default user account.
 - v. In the **Database** field, enter the database name.
8. Optional: By default, pipeline definitions are stored as Kubernetes resources, enabling version control, GitOps workflows, and integration with OpenShift GitOps or similar tools. To store pipeline definitions in the internal database instead, clear the **Store pipeline definitions in Kubernetes** checkbox in the **Pipeline definition storage** section.
 9. Optional: By default, caching is configurable at both the pipeline and task levels. To disable caching for all pipelines and tasks in the pipeline server and override any pipeline-level and task-level caching settings, clear the **Allow caching to be configured per pipeline and task** checkbox in the **Pipeline caching** section.
 10. Click **Configure pipeline server**.

Verification

On the **Pipelines** tab for the project:

- The **Import pipeline** button is available.
- When you click the action menu () and then click **Manage pipeline server configuration**, the pipeline server details are displayed.

Additional resources

- [Overview of pipelines caching](#)

1.1.1. Configure a pipeline server with an external Amazon RDS database

To configure a pipeline server with an external Amazon Relational Database Service (RDS) database, you must configure OpenShift AI to trust the certificates issued by its certificate authorities (CA).



IMPORTANT

If you are configuring a pipeline server for production pipeline workloads, Red Hat recommends that you use an external MySQL or MariaDB database.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have logged in to Red Hat OpenShift AI.
- You have created a project that you can add a pipeline server to.
- You have an existing S3-compatible object storage bucket, and you have configured your storage account with write access to your S3 bucket.

Procedure

1. Before configuring your pipeline server, from [Amazon RDS: Certificate bundles by AWS Region](#) , download the PEM certificate bundle for the region that the database was created in. For example, if the database was created in the **us-east-1** region, download **us-east-1-bundle.pem**.
2. In a terminal window, log in to the OpenShift cluster where OpenShift AI is deployed.

```
oc login api.<cluster_name>.<cluster_domain>:6443 --web
```

3. Run the following command to fetch the current OpenShift AI trusted CA configuration and store it in a new file:

```
oc get dscinitializations.dscinitialization.opendatahub.io default-dsci -o json | jq '.spec.trustedCABundle.customCABundle' > /tmp/my-custom-ca-bundles.crt
```

4. Run the following command to append the PEM certificate bundle that you downloaded to the new custom CA configuration file:

```
cat us-east-1-bundle.pem >> /tmp/my-custom-ca-bundles.crt
```

5. Run the following command to update the OpenShift AI trusted CA configuration to trust certificates issued by the CAs included in the new custom CA configuration file:

```
oc patch dscinitialization default-dsci --type='json' -p='[{"op":"replace","path":"/spec/trustedCABundle/customCABundle","value":""$(awk '{printf "%s\\n", $0}' /tmp/my-custom-ca-bundles.crt)"}]'
```

6. Configure a pipeline server, as described in [Configuring a pipeline server](#).

Verification

- The pipeline server starts successfully.
- You can import and run AI pipelines.

1.2. OPTIONS FOR DEFINING A PIPELINE

The Kubeflow Pipelines SDK enables you to define end-to-end machine learning and AI pipelines. Use the latest Kubeflow Pipelines 2.0 SDK to build your AI pipeline in Python code. After you have built your pipeline, use the SDK to compile it into an Intermediate Representation (IR) YAML file. For more information about compiling pipelines, see *Compiling the pipeline YAML with the Kubeflow Pipelines SDK* and *Compiling Kubernetes-native manifests with the Kubeflow Pipelines SDK*. Compiling to Kubernetes-native manifests is optional and applies only when your pipeline server is configured to use Kubernetes API storage. After defining the pipeline, you can import the YAML file to the OpenShift AI dashboard to enable you to configure its execution settings.



IMPORTANT

If you are using OpenShift AI on a cluster running in FIPS mode, any custom container images for AI pipelines must be based on UBI 9 or RHEL 9. This ensures compatibility with FIPS-approved pipeline components and prevents errors related to mismatched OpenSSL or GNU C Library (glibc) versions.

You can also use the Elyra JupyterLab extension to create and run AI pipelines within JupyterLab. For more information about creating pipelines in JupyterLab, see [Working with pipelines in JupyterLab](#). For more information about the Elyra JupyterLab extension, see [Elyra Documentation](#).

Additional resources

- [Kubeflow Pipelines 2.0 Documentation](#)
- [Elyra Documentation](#)

1.2.1. Compile the pipeline YAML with the Kubeflow Pipelines SDK

Before you can define your pipeline in the cluster, you must convert your Python-defined pipeline into YAML format. You can use the Kubeflow Pipelines (KFP) Software Development Kit (SDK) to compile your pipeline code into a deployable YAML file for declarative GitOps deployment.

Prerequisites

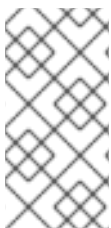
- You have installed Python 3.11 or later in your local environment.
- You have installed the Kubeflow Pipelines SDK package (**kfp**) version 2.14.3 or later.
- You have a valid Python pipeline definition file.

Procedure

Compile your pipeline by using the KFP SDK to generate the pipeline YAML file.

In the following example, replace `<pipeline_file>.py` with the name of your Python pipeline file and specify an output file for the compiled YAML:

```
$ kfp dsl compile \
  --py <pipeline_file>.py \
  --output <compiled_pipeline_file>.yaml
```



NOTE

The generated `<compiled_pipeline_file>.yaml` file contains the compiled pipeline specification in YAML format. You can use this content as the value of the **pipelineSpec** field when you create a **PipelineVersion** custom resource (CR). You can also store the file in Git for declarative or GitOps-based deployment.

Verification

Verify that the generated file includes a **pipelineSpec** key followed by the compiled pipeline definition:

```
$ head -n 10 <compiled_pipeline_file>.yaml
```

Additional resources

- [Compiling a pipeline with the Kubeflow Pipelines SDK](#)

1.2.2. Compile Kubernetes-native manifests with the Kubeflow Pipelines SDK

If your pipeline server uses the Kubernetes native API mode, you can compile your pipeline directly to Kubernetes manifests. The output includes **Pipeline** and **PipelineVersion** custom resources with **spec.pipelineSpec** and, when you use Kubernetes resource configuration, an optional **spec.platformSpec**.

Prerequisites

- You have installed Python 3.11 or later in your local environment.
- You have installed the Kubeflow Pipelines SDK package (**kfp**) version 2.14.3 or later.
- You have a valid Python pipeline definition file.

Procedure

1. Save the following code as a new file named **compile.py** in your working directory. The example uses the **KubernetesManifestOptions** class from the **kfp.compiler.compiler_utils** module to define pipeline metadata such as the name, version, and namespace.

Example compile script

```
from kfp import dsl, compiler
from kfp.compiler.compiler_utils import KubernetesManifestOptions

@dsl.pipeline(name="<pipeline_name>")
def my_pipeline():
    pass # define your tasks

compiler.Compiler().compile(
    pipeline_func=my_pipeline,
    package_path="<output_file>.yaml",
    kubernetes_manifest_format=True,
    kubernetes_manifest_options=KubernetesManifestOptions(
        pipeline_name="<pipeline_name>",
        pipeline_version_name="<version_name>",
        namespace="<namespace>",
        include_pipeline_manifest=True,
    ),
)
```

2. Run the script to compile your pipeline and generate the Kubernetes manifests:

```
$ python compile.py
```

Verification

Verify that the compiled output includes the expected resources:

```
apiVersion: pipelines.kubeflow.org/v2beta1
kind: Pipeline
---
apiVersion: pipelines.kubeflow.org/v2beta1
kind: PipelineVersion
```

```
spec:
  pipelineSpec: ...
  platformSpec: ... # present when Kubernetes resource configuration is used
```

Additional resources

- [Compiling for Kubernetes native API mode](#)

1.2.3. Authenticate the Kubeflow Pipelines SDK with a pipeline server

You can connect the Kubeflow Pipelines (KFP) SDK to a pipeline server that is exposed by OpenShift AI. The pipeline server route is protected by OpenShift OAuth, so you must provide a valid access token when you create the KFP client.

Prerequisites

- You have logged in to the OpenShift CLI (**oc**) as a user who can access the project.
- You have created a project and configured a pipeline server for that project.
- You have installed Python and the required packages in your environment.
- You have installed the Kubeflow Pipelines SDK. For more information, see [Installing the Kubeflow Pipelines SDK](#).
- Optional: If your cluster uses a custom or self-signed certificate, you know the path to the trusted certificate bundle that your environment uses.

Procedure

1. Set environment variables for your project and pipeline server route:

```
export NAMESPACE=<project_namespace>
export DSPA_NAME=$(oc -n "$NAMESPACE" get dspa -o
jsonpath='{.items[0].metadata.name}')
export API_URL="https://$(oc -n "$NAMESPACE" get route "ds-pipeline-${DSPA_NAME}" -o
jsonpath='{.spec.host}')
```

Replace **<project_namespace>** with the name of your project.

2. Obtain an OpenShift access token for the current user:

```
export OCP_TOKEN=$(oc whoami --show-token)
```




NOTE

Avoid pasting the access token directly into commands or scripts. The token can appear in your shell history or in process listings if you pass it as a literal argument.

To reduce this risk, store the token in an environment variable and reference it from your code or commands. For example:

```
./venv/bin/python my_script.py --kfp-server-host "$API_URL" --namespace
"$NAMESPACE" --token "$OCP_TOKEN"
```

Alternatively, use a prompt with **read -s** to input the token securely at runtime.

- Optional: If you are running outside the cluster or you use a custom or self-signed certificate, set an environment variable for your trusted certificate bundle:

```
export SSL_CA_CERT=/etc/pki/tls/custom-certs/ca-bundle.crt
```

Adjust the path if your environment uses a different certificate location.

- In your Python environment, create a KFP client that uses the pipeline server route and OpenShift access token:

```
import os
from kfp.client import Client

api_url = os.environ["API_URL"]
token = os.environ["OCP_TOKEN"]
namespace = os.environ["NAMESPACE"]

# Optional: Use a custom certificate bundle if required
ssl_ca_cert = os.environ.get("SSL_CA_CERT", None)

client_args = {
    "host": api_url,
    "existing_token": token,
    "namespace": namespace,
}

if ssl_ca_cert:
    client_args["ssl_ca_cert"] = ssl_ca_cert

client = Client(**client_args)
```

You can customize the client configuration by adding additional values to **client_args**, such as **timeout**, **retry_count**, or **verify_ssl**.

- Verify the connection by calling the API. For example, list experiments or pipelines:

```
print(client.list_experiments())
# or
print(client.list_pipelines())
```

Verification

- The Python code runs without authentication errors.
- The command output lists experiments or pipelines that are defined on the pipeline server for the specified project.

Next steps

- Use the KFP SDK to compile and upload pipelines, create pipeline runs, or manage pipeline versions against the authenticated pipeline server.
- If required, integrate this client configuration into your own automation scripts or external applications that orchestrate pipelines on OpenShift AI.

1.2.4. Define a pipeline by using the Kubernetes API

You can define AI pipelines and pipeline versions by using the Kubernetes API, which stores them as custom resources in the cluster instead of the internal database. This approach makes it easier to use OpenShift GitOps (Argo CD) or similar tools to manage pipelines and pipeline versions, while still allowing you to manage them through the OpenShift AI dashboard, API, and the Kubeflow Pipelines (KFP) Software Development Kit (SDK). You can generate the required manifests by using the Kubeflow Pipelines SDK; see *Compiling the pipeline YAML with the Kubeflow Pipelines SDK* or *Compiling Kubernetes-native manifests with the Kubeflow Pipelines SDK*.



NOTE

If your pipeline server is already configured to use Kubernetes API storage, you can still use the OpenShift AI dashboard and REST API to view pipeline details, run pipelines, and create schedules. In this mode, the Kubernetes API acts as the storage backend, so your existing tools continue to work as expected.

Prerequisites

- You have OpenShift AI administrator privileges or you are the project owner.
- You have a project with a running pipeline server.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- If you plan to create a **PipelineVersion** custom resource, you have either:
 - Compiled your Python pipeline to IR YAML by using the KFP SDK. See *Compiling the pipeline YAML with the Kubeflow Pipelines SDK*.
 - Compiled Kubernetes-native manifests by using the KFP SDK. See *Compiling Kubernetes-native manifests with the Kubeflow Pipelines SDK*.

Procedure

1. In a terminal window, log in to your OpenShift cluster by using the OpenShift CLI (**oc**):

```
$ oc login -u <user_name>
```

When prompted, enter the OpenShift server URL, connection type, and your password.

- To configure the pipeline server to use Kubernetes API storage instead of the default **database** option, set the **spec.apiServer.pipelineStore** field to **kubernetes** in your project's **DataSciencePipelinesApplication** (DSPA) custom resource.

In the following command, replace *<dspa_name>* with the name of your DSPA custom resource, and replace *<namespace>* with the name of your project:

```
$ oc patch dspa <dspa_name> -n <namespace> \
  --type=merge \
  -p {"spec": {"apiServer": {"pipelineStore": "kubernetes"}}}
```



WARNING

When you switch the pipeline server from database storage to Kubernetes API storage, existing pipelines that were stored in the internal database are no longer visible in the OpenShift AI dashboard or REST API. To view or manage those pipelines again, change the **spec.apiServer.pipelineStore** field back to **database**.

- Define a **Pipeline** custom resource in a YAML file with the following contents:

Example pipeline definition

```
apiVersion: pipelines.kubeflow.org/v2beta1
kind: Pipeline
metadata:
  name: <name>
  namespace: <namespace>
spec:
  displayName: <displayName>
```

- **name:** The immutable Kubernetes resource name of your pipeline.
 - **namespace:** The name of your project.
 - **displayName:** The user-friendly display name of your pipeline, which is shown in the dashboard and REST API.
- Apply the pipeline definition to create the **Pipeline** custom resource in your cluster. In the following command, replace *<pipeline_yaml_file>* with the name of your YAML file:

Example command

```
$ oc apply -f <pipeline_yaml_file>.yaml
```

- Alternatively, if you compiled Kubernetes-native manifests with the KFP SDK, you can apply the generated file directly without manually creating separate YAML files:

```
$ oc apply -f <output_file>.yaml
```

The generated file includes both **Pipeline** and **PipelineVersion** resources. You can skip the following manual definition steps and proceed to the verification step.

- Define a **PipelineVersion** custom resource in a YAML file with the following contents:

Example pipeline version definition

```
apiVersion: pipelines.kubeflow.org/v2beta1
kind: PipelineVersion
metadata:
  name: <name>
  namespace: <namespace>
spec:
  pipelineName: <pipelineName>
  displayName: <displayName>
  description: This is the first version of the pipeline.
  pipelineSpec:
    # ... YAML generated by compiling Python pipeline with KFP SDK ...
```

- **name:** The name of your pipeline version.
 - **namespace:** The name of your project.
 - **pipelineName:** The immutable Kubernetes resource name of your pipeline. This value must match the **metadata.name** value in the **Pipeline** custom resource.
 - **displayName:** The user-friendly display name of your pipeline version, which is shown in the dashboard and REST API.
 - **pipelineSpec:** The YAML content that you generated by using the Kubeflow Pipelines (KFP) SDK.
- Apply the pipeline version definition to create the **PipelineVersion** custom resource in your cluster.
In the following command, replace *<pipeline_version_yaml_file>* with the name of your YAML file:

Example command

```
$ oc apply -f <pipeline_version_yaml_file>.yaml
```

After creating the pipeline version, the system automatically applies the following labels to the pipeline version for easier filtering:

Example automatic labels

```
pipelines.kubeflow.org/pipeline-id: <metadata.uid of the pipeline>
pipelines.kubeflow.org/pipeline: <pipeline name>
```

Verification

1. Check that the **Pipeline** custom resource was successfully created:

```
$ oc get pipeline <pipeline_name> -n <namespace>
```

2. Check that the **PipelineVersion** custom resource was successfully created:

```
$ oc get pipelineversion <pipeline_version_name> -n <namespace>
```

1.2.5. Migrate pipelines from database to Kubernetes API storage

You can migrate existing pipelines and pipeline versions from the internal database to Kubernetes custom resources. This makes it easier to use OpenShift GitOps (Argo CD) or similar tools to manage pipelines and pipeline versions, while still allowing you to manage them through the OpenShift AI dashboard, API, and the Kubeflow Pipelines (KFP) Software Development Kit (SDK).

This procedure uses a community-supported Kubeflow Pipelines migration script to export pipelines from the AI Pipelines API and generate corresponding **Pipeline** and **PipelineVersion** custom resources for import into your cluster.



IMPORTANT

The migration script in this procedure is maintained by the Kubeflow Pipelines community and is not supported by Red Hat. Before you use the script, review the repository and validate it in a non-production environment.



WARNING

The pipeline and pipeline version IDs change during migration, so existing pipeline runs do not map to the migrated pipeline version. The original ID is stored in the **`pipelines.kubeflow.org/original-id`** label.

Prerequisites

- You have OpenShift AI administrator privileges or you are the project owner.
- You have a project with a running pipeline server.
- The pipeline server is configured with **`spec.apiServer.pipelineStore: database`**.
- You have Python 3.11 installed in your local environment.
- You have installed the OpenShift CLI (**`oc`**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. In a terminal window, log in to your OpenShift cluster by using the OpenShift CLI (**oc**):

```
$ oc login -u <user_name>
```

When prompted, enter the OpenShift server URL, connection type, and your password.

2. Set environment variables for your project and get the pipeline API route.
In the **export** command, replace *<namespace>* with the name of your project:

```
echo "Setting the prerequisite variables"
export NAMESPACE=<namespace>
export DSPA_NAME=$(oc -n $NAMESPACE get dspa -o
jsonpath={.items[0].metadata.name})
export API_URL="https://$(oc -n $NAMESPACE get route "ds-pipeline-$DSPA_NAME" -o
jsonpath={.spec.host})"
```

3. Create a Python virtual environment and install the required dependencies.

```
echo "Set up the Python prerequisites"
python3.11 -m venv .venv
./.venv/bin/pip install kfp requests PyYAML
```

4. Download and run the Kubeflow Pipelines community migration script.
The script connects to the AI Pipelines API, exports all pipelines and versions from the specified project, and generates one YAML file per pipeline in a local **kfp-exported-pipelines/** directory. Each file includes a **Pipeline** resource followed by all associated **PipelineVersion** resources.

- a. Run the following command:

```
curl -L https://raw.githubusercontent.com/kubeflow/pipelines/refs/heads/master/tools/k8s-
native/migration.py -o migration.py
./.venv/bin/python migration.py --skip-tls-verify --kfp-server-host $API_URL --namespace
$NAMESPACE --token "$(oc whoami --show-token)"
```

NOTE

The **--skip-tls-verify** option disables certificate validation and should be used only in development environments or when connecting to a server with a self-signed certificate. In production environments, provide a valid certificate bundle instead.

Additionally, passing the access token directly on the command line might expose it in shell history or process lists. To reduce this risk, store the token in an environment variable and reference it in your command:

```
export KFP_TOKEN=$(oc whoami --show-token)
./.venv/bin/python migration.py --kfp-server-host $API_URL --namespace
$NAMESPACE --token "$KFP_TOKEN"
```

Alternatively, use a prompt with **read -s** to input the token securely at runtime.

- b. Optional: For more information about the script, run the following command:

```
./venv/bin/python migration.py --help
```

- c. If you plan to create new or updated **PipelineVersion** custom resources after migration, you can compile your pipeline code by using the Kubeflow Pipelines SDK. For more information, see *Compiling the pipeline YAML with the Kubeflow Pipelines SDK* and *Compiling Kubernetes-native manifests with the Kubeflow Pipelines SDK*.

5. Apply the exported Kubernetes custom resources to your cluster.

```
oc apply -f ./kfp-exported-pipelines
```

6. Change the pipeline server to use Kubernetes API storage.

```
oc -n "$NAMESPACE" patch dsps "$DSPA_NAME" --type=merge -p {"spec":{"apiServer":{"pipelineStore":"kubernetes"}}
```



NOTE

To view pipelines that were stored in the internal database and not migrated, you can temporarily change the pipeline server back to **database** storage.

```
oc -n $NAMESPACE patch dsps $DSPA_NAME --type=merge -p {"spec":{"apiServer":{"pipelineStore":"database"}}
```

7. Repeat this procedure for each additional project that you want to migrate, changing **NAMESPACE** to the appropriate project name.
8. Optional: Clean up the local environment.

```
rm -rf .venv migration.py
```

Verification

1. Check that the **Pipeline** and **PipelineVersion** custom resources were created in your project:

```
$ oc -n <namespace> get pipelines.pipelines.kubeflow.org
$ oc -n <namespace> get pipelineversions.pipelines.kubeflow.org
```

2. Verify that the pipeline server is using Kubernetes API storage:

```
$ oc -n <namespace> get dsps <dsps_name> -o jsonpath={.spec.apiServer.pipelineStore}
{"n"}
```

The command should return **kubernetes**.

Additional resources

- [Kubeflow Pipelines community migration script source](#)
- [Script README and usage details](#)

1.3. IMPORT A PIPELINE TO THE DASHBOARD

To help you begin working with AI pipelines in OpenShift AI, you can import a YAML file containing your pipeline's code to an active pipeline server, or you can import the YAML file from a URL. This file contains a Kubeflow pipeline compiled by using the Kubeflow compiler. After you have imported the pipeline to a pipeline server, you can execute the pipeline by creating a pipeline run.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a configured pipeline server.
- You have compiled your pipeline with the Kubeflow compiler and you have access to the resulting YAML file.
- If you are uploading your pipeline from a URL, the URL is publicly accessible.



NOTE

If your pipeline is defined in Python code instead of a YAML file, compile it first by using the KFP SDK. For more information, see *Compiling the pipeline YAML with the Kubeflow Pipelines SDK*.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Pipeline definitions**.
2. On the **Pipeline definition** page, from the **Project** drop-down list, select the project that you want to import a pipeline to.
3. Click **Import pipeline**.
4. In the **Import pipeline** dialog, enter the details for the pipeline that you want to import.
 - a. In the **Pipeline name** field, enter a name for the pipeline that you want to import.
 - b. In the **Pipeline description** field, enter a description for the pipeline that want to import.
 - c. Select where you want to import your pipeline from by performing one of the following actions:
 - Select **Upload a file** to upload your pipeline from your local machine's file system. Import your pipeline by clicking **Upload**, or by dragging and dropping a file.
 - Select **Import by url** to upload your pipeline from a URL, and then enter the URL into the text box.
 - d. Click **Import pipeline**.

Verification

- The pipeline that you imported is displayed on the **Pipeline definitions** page and on the **Pipelines** tab on the project details page.

1.4. DELETE A PIPELINE

If you no longer require access to your AI pipeline on the dashboard, you can delete it so that it does not appear on the **Pipeline definitions** page. .Prerequisites * You have logged in to Red Hat OpenShift AI. * There are active pipelines available on the **Pipeline definitions** page. * The pipeline that you want to delete does not contain any pipeline versions. * The pipeline that you want to delete does not contain any pipeline versions. For more information, see [Deleting a pipeline version](#).

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Pipeline definitions**.
2. On the **Pipeline definitions** page, from the **Project** drop-down list, select the project that contains the pipeline that you want to delete.
3. Click the action menu () beside the pipeline that you want to delete, and then click **Delete pipeline**.
4. In the **Delete pipeline** dialog, enter the pipeline name in the text field to confirm that you intend to delete it.
5. Click **Delete pipeline**.

Verification

- The AI pipeline that you deleted is no longer displayed on the **Pipeline definitions** page.

1.5. DELETE A PIPELINE SERVER

After you have finished running your AI pipelines, you can delete the pipeline server. Deleting a pipeline server automatically deletes all of its associated pipelines, pipeline versions, and runs. If your pipeline data is stored in a database, the database is also deleted along with its meta-data. In addition, after deleting a pipeline server, you cannot create new pipelines or pipeline runs until you create another pipeline server.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a pipeline server.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Pipeline definitions**.
2. On the **Pipeline definitions** page, from the **Project** drop-down list, select the project that contains the pipeline server that you want to delete.
3. From the **Pipeline server actions** list, select **Delete pipeline server**.
4. In the **Delete pipeline server** dialog, enter the name of the pipeline server in the text field to confirm that you intend to delete it.
5. Click **Delete**.

Verification

- Pipelines previously assigned to the deleted pipeline server no longer appear on the **Pipeline definitions** page for the relevant project.
- Pipeline runs previously assigned to the deleted pipeline server no longer appear on the **Runs** page for the relevant project.

1.6. VIEW PIPELINE SERVER DETAILS

You can view the details of pipeline servers configured in OpenShift AI, such as the pipeline's connection details and where its data is stored.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that contains an active and available pipeline server.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Pipeline definitions**.
2. On the **Pipeline definitions** page, from the **Project** drop-down list, select the project that contains the pipeline server that you want to view.
3. From the **Pipeline server actions** list, select **Manage pipeline server configuration**.

Verification

- You can view the pipeline server details in the **Manage pipeline server** dialog.

1.7. VIEW EXISTING PIPELINES

You can view the details of pipelines that you have imported to Red Hat OpenShift AI, such as the pipeline's last run, when it was created, the pipeline's executed runs, and details of any associated pipeline versions.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a pipeline server.
- You have imported a pipeline to an active pipeline server.
- Existing pipelines are available.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Pipeline definitions**.
2. On the **Pipeline definitions** page, from the **Project** drop-down list, select the project that contains the pipelines that you want to view.

- Optional: Click **Expand** () on the row of a pipeline to view its pipeline versions.

Verification

- A list of pipelines is displayed on the **Pipeline definitions** page.

1.8. PIPELINE VERSIONS

You can manage incremental changes to pipelines in OpenShift AI by using versioning. This allows you to develop and deploy pipelines iteratively, preserving a record of your changes. You can track and manage your changes on the OpenShift AI dashboard, allowing you to schedule and execute runs against all available versions of your pipeline.

1.9. UPLOAD A PIPELINE VERSION

You can upload a YAML file to an active pipeline server that contains the latest version of your pipeline, or you can upload the YAML file from a URL. The YAML file must consist of a Kubeflow pipeline compiled by using the Kubeflow compiler. After you upload a pipeline version to a pipeline server, you can execute it by creating a pipeline run.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a configured pipeline server.
- You have a pipeline version available and ready to upload.
- If you are uploading your pipeline version from a URL, the URL is publicly accessible.
- If your pipeline version is based on Python code, compile it to YAML before uploading. For more information, see *Compiling the pipeline YAML with the Kubeflow Pipelines SDK*.

Procedure

- From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Pipeline definitions**.
- On the **Pipeline definitions** page, from the **Project** drop-down list, select the project that you want to upload a pipeline version to.
- Click the **Import pipeline** drop-down list, and then select **Upload new version**.
- In the **Upload new version** dialog, enter the details for the pipeline version that you are uploading.
 - From the **Pipeline** list, select the pipeline that you want to upload your pipeline version to.
 - In the **Pipeline version name** field, confirm the name for the pipeline version, and change it if necessary.
 - In the **Pipeline version description** field, enter a description for the pipeline version.
 - Select where you want to upload your pipeline version from by performing one of the following actions:
 - Select **Upload a file** to upload your pipeline version from your local machine's file

system. Import your pipeline version by clicking **Upload**, or by dragging and dropping a file.

- Select **Import by url** to upload your pipeline version from a URL, and then enter the URL into the text box.

e. Click **Upload**.

Verification

- The pipeline version that you uploaded is displayed on the **Pipeline definitions** page. Click **Expand** () on the row containing the pipeline to view its versions.
- The **Version** column on the row containing the pipeline version that you uploaded on the **Pipeline definitions** page increments by one.

1.10. DELETE A PIPELINE VERSION

You can delete specific versions of a pipeline when you no longer require them. Deleting a default pipeline version automatically changes the default pipeline version to the next most recent version. If no pipeline versions exist, the pipeline persists without a default version.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a pipeline server.
- You have imported a pipeline to an active pipeline server.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Pipeline definitions**. The **Pipeline definitions** page opens.
2. Delete the pipeline versions that you no longer require:
 - To delete a single pipeline version:
 - a. From the **Project** list, select the project that contains a version of a pipeline that you want to delete.
 - b. On the row containing the pipeline, click **Expand** ().
 - c. Click the action menu () beside the version that you want to delete, and then click **Delete pipeline version**. The **Delete pipeline version** dialog opens.
 - d. Enter the name of the pipeline version in the text field to confirm that you intend to delete it.
 - e. Click **Delete**.
 - To delete multiple pipeline versions:

- a. On the row containing each pipeline version that you want to delete, select the checkbox.
- b. Click the action menu () next to the **Import pipeline** drop-down list, and then select **Delete** from the list.

Verification

- The pipeline version that you deleted is no longer displayed on the **Pipeline definitions** page or on the **Pipelines** tab for the project.

1.11. VIEW PIPELINE VERSION DETAILS

You can view the details of a pipeline version that you have uploaded to Red Hat OpenShift AI, such as its graph and YAML code.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a pipeline server.
- You have a pipeline available on an active and available pipeline server.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Pipeline definitions**.
2. On the **Pipeline definitions** page, from the **Project** drop-down list, select the project that contains the pipeline versions that you want to view details for.
3. Click the pipeline name to view further details of its most recent version. The pipeline version details page opens, displaying the **Graph**, **Summary**, and **Pipeline spec** tabs.

Alternatively, click **Expand** () on the row containing the pipeline that you want to view versions for, and then click the pipeline version that you want to view the details of. The pipeline version details page opens, displaying the **Graph**, **Summary**, and **Pipeline spec** tabs.

Verification

- On the pipeline version details page, you can view the pipeline graph, summary details, and YAML code.

1.12. DOWNLOAD A PIPELINE VERSION

To make further changes to an AI pipeline version that you previously uploaded to OpenShift AI, you can download pipeline version code from the user interface.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a configured pipeline server.

- You have created and imported a pipeline to an active pipeline server that is available to download.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Pipeline definitions**.
2. On the **Pipeline definitions** page, from the **Project** drop-down list, select the project that contains the version that you want to download.
3. Click **Expand** () beside the pipeline that contains the version that you want to download.
4. Click the pipeline version that you want to download.
The pipeline version details page opens.
5. Click the **Pipeline spec** tab, and then click the **Download** button () to download the YAML file that contains the pipeline version code to your local machine.

Verification

- The pipeline version code downloads to your browser's default directory for downloaded files.

1.13. PIPELINE CACHING

You can use caching within AI pipelines to optimize execution times and improve resource efficiency. Caching reduces redundant task execution by reusing results from previous runs with identical inputs.

Caching is particularly beneficial for iterative tasks, where intermediate steps might not need to be repeated. Understanding caching can help you design more efficient pipelines and save time in model development.

Caching operates by storing the outputs of successfully completed tasks and comparing the inputs of new tasks against previously cached ones. If a match is found, OpenShift AI reuses the cached results instead of re-executing the task, reducing computation time and resource usage.

1.13.1. Caching criteria

For caching to be effective, the following criteria determine if a task can use previously cached results:

- **Input data and parameters** If the input data and parameters for a task are unchanged from a previous run, cached results are eligible for reuse.
- **Task code and configuration** Changes to the task code or configurations invalidate the cache to ensure that modifications are always reflected.
- **Pipeline environment**: Changes to the pipeline environment, such as dependency versions, also affect caching eligibility to maintain consistency.

1.13.2. Viewing cached steps in the OpenShift AI user interface

Cached steps in pipelines are visually indicated in the user interface (UI):

- Tasks that use cached results display a green icon, helping you quickly identify which steps were cached. The **Status** field in the side panel displays **Cached** for cached tasks.

- The UI also includes information about when the task was previously executed, allowing for easy verification of cache usage.

To check the caching status of specific tasks, navigate to the pipeline details view in the UI. Cached and non-cached tasks are clearly indicated. Cached tasks do not display execution logs because they reuse previously generated outputs and are not re-executed.

1.13.3. Control caching in pipelines

Caching is enabled by default in OpenShift AI to improve performance. However, there are instances when disabling caching might be necessary for specific tasks, an entire pipeline, or all pipelines. For example, caching might not be beneficial for tasks that rely on frequently updated data or unique computational needs. In other cases, such as debugging, development, or when deterministic re-execution is required, you might want to disable caching for all pipelines.

CAUTION

Disabling caching at the pipeline or pipeline server level causes all tasks to re-run, potentially increasing compute time and resource usage.

You can control caching for AI pipelines in the following ways:

- **Individual task:** Data scientists can disable caching for specific steps in a pipeline.
- **Pipeline (submit time):** Data scientists can disable caching when submitting a pipeline run.
- **Pipeline (compile time):** Data scientists can disable caching when compiling a pipeline.
- **All pipelines (pipeline server):** You can disable caching for all pipelines in the pipeline server, which overrides all pipeline and task-level caching settings.

1.13.3.1. Disabling caching for individual tasks

To disable caching for a particular task, apply the **set_caching_options** method directly to the task in your pipeline code:

```
task_name.set_caching_options(False)
```

After applying this setting, OpenShift AI runs the task in future pipeline runs, ignoring any cached results.

You can re-enable caching for individual tasks by setting the **set_caching_options** parameter to **True** or by omitting **set_caching_options**.

This setting is ignored if caching is disabled in the pipeline server.

1.13.3.2. Disabling caching for a pipeline at submit time

To disable caching for the entire pipeline during pipeline submission, set the **enable_caching** parameter to **False** in your pipeline code. This setting ensures that no steps are cached during pipeline execution. The **enable_caching** parameter is available only when using the **kfp.client** to submit pipelines or start pipeline runs, such as the **run_pipeline** method.

Example:

```
import kfp
client = kfp.Client()
client.run_pipeline(
    experiment_id=experiment.id,
    pipeline_id=pipeline.id,
    job_name="no-cache-run",
    params={},          # optional
    enable_caching=False,
)
```

This setting is ignored if caching is disabled during pipeline compilation or in the pipeline server.

1.13.3.3. Disabling caching for a pipeline at compile time

To disable caching for the entire pipeline during compilation, set one of the following options in your local environment or workbench:

- Environment variable:

```
export KFP_DISABLE_EXECUTION_CACHING_BY_DEFAULT=true
```

- CLI flag (when using **kfp dsl compile**):

```
kfp dsl compile --disable-execution-caching-by-default
```

These settings are ignored if caching is disabled in the pipeline server.

1.13.3.4. Disabling caching for all pipelines (pipeline server)

To disable caching for all pipelines in the pipeline server and override all pipeline and task-level caching settings, use either of the following methods:

Pipeline server configuration

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Pipeline definitions**.
2. On the **Pipeline definitions** page, from the **Project** drop-down list, select the project that contains the pipeline server that you want to configure.
3. From the **Pipeline server actions** list, select **Manage pipeline server configuration**.
4. In the **Pipeline caching** section, clear the **Allow caching to be configured per pipeline and task** checkbox.
5. Click **Save**.

DataSciencePipelinesApplication (cluster administrator)

In the OpenShift console or CLI, set the **cacheEnabled** field to **false** in the **DataSciencePipelinesApplication** (DSPA) custom resource for the project.

Example:

```
apiVersion: datasciencepipelinesapplications.opendatahub.io/v1
kind: DataSciencePipelinesApplication
metadata:
```



```

name: my-dspa
namespace: my-namespace
spec:
  apiServer:
    cacheEnabled: false

```

To allow caching to be configured at the pipeline and task level, set the **cacheEnabled** field to **true** in the DSPA custom resource.

After applying this setting, all pipeline and task-level caching settings are ignored.



NOTE

Changing this setting updates the **CACHEENABLED** environment variable in the pipeline server deployment.

Verification

After configuring caching settings, you can verify its behavior by using one of the following methods:

- **Check the UI** Locate the green icons in the task list to identify cached steps.
- **Test task re-runs:** Disable caching on specific tasks or the pipeline to confirm that steps re-execute as expected.
- **Validate inputs:** Ensure the task inputs, parameters, and runtime settings are unchanged when caching is applied.



NOTE

You can also disable caching for a single node or for your entire pipeline in JupyterLab using Elyra. For more information, see [Disabling node caching in Elyra](#).

Additional resources

- Kubeflow caching documentation: [Kubeflow Pipelines - Use Caching](#)
- The **kfp.client** module documentation for the **enable_caching** parameter: [KFP SDK API Reference - kfp.client](#)

CHAPTER 2. MANAGE PIPELINE EXPERIMENTS

2.1. PIPELINE EXPERIMENTS FOR ORGANIZED RUN COMPARISON

A pipeline experiment is a workspace where you can try different configurations of your pipelines. You can use experiments to organize your runs into logical groups. As a data scientist, you can use OpenShift AI to define, manage, and track pipeline experiments. You can view a record of previously created and archived experiments from the **Experiments** page in the OpenShift AI user interface. Pipeline experiments contain pipeline runs, including recurring runs. This allows you to try different configurations of your pipelines.

When you work with AI pipelines, it is important to monitor and record your pipeline experiments to track the performance of your AI pipelines. You can compare the results of up to 10 pipeline runs at one time, and view available parameter, scalar metric, confusion matrix, and receiver operating characteristic (ROC) curve data for all selected runs.

You can view artifacts for an executed pipeline run from the OpenShift AI dashboard. Pipeline artifacts can help you to evaluate the performance of your pipeline runs and make it easier to understand your pipeline components. Pipeline artifacts can range from plain text data to detailed, interactive data visualizations.

2.2. CREATING A PIPELINE EXPERIMENT

Pipeline experiments are workspaces where you can try different configurations of your pipelines. You can also use experiments to organize your pipeline runs into logical groups. Pipeline experiments contain pipeline runs, including recurring runs.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a configured pipeline server.
- You have imported a pipeline to an active pipeline server.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Experiments**.
2. On the **Experiments** page, from the **Project** drop-down list, select the project to create the pipeline experiment in.
3. Click **Create experiment**.
4. In the **Create experiment** dialog, configure the pipeline experiment:
 - a. In the **Experiment name** field, enter a name for the pipeline experiment.
 - b. In the **Description** field, enter a description for the pipeline experiment.
 - c. Click **Create experiment**.

Verification

- The pipeline experiment that you created is displayed on the **Experiments** tab.

2.3. ARCHIVING A PIPELINE EXPERIMENT

You can archive your pipeline experiments to store and retain records for future reference. If you need to reuse an archived experiment, you can restore it at any time. Deleting pipeline experiments is a separate action and happens only when you explicitly choose to delete them. Unarchived experiments remain stored unless you manually delete them.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and has a pipeline server.
- You have imported a pipeline to an active pipeline server.
- A pipeline experiment is available to archive.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Experiments**.
2. On the **Experiments** page, from the **Project** drop-down list, select the project that contains the pipeline experiment that you want to archive.
3. Click the action menu () beside the pipeline experiment that you want to archive, and then click **Archive**.
4. In the **Archiving experiment** dialog, enter the pipeline experiment name in the text field to confirm that you intend to archive it.
5. Click **Archive**.

Verification

- The archived pipeline experiment does not appear on the **Experiments** tab, and instead is displayed on the **Archive** tab on the **Experiments** page for the pipeline experiment.

2.4. DELETING AN ARCHIVED PIPELINE EXPERIMENT

You can delete pipeline experiments from the OpenShift AI experiment archive.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a configured pipeline server.
- You have imported a pipeline to an active pipeline server.
- A pipeline experiment is available in the pipeline archive.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Experiments**.

2. On the **Experiments** page, from the **Project** drop-down list, select the project that contains the archived pipeline experiment that you want to delete.
3. Click the **Archive** tab.
4. Click the action menu () beside the pipeline experiment that you want to delete, and then click **Delete**.
5. In the **Delete experiment?** dialog, enter the pipeline experiment name in the text field to confirm that you intend to delete it.
6. Click **Delete**.

Verification

- The pipeline experiment that you deleted is no longer displayed on the **Archive** tab on the **Experiments** page.

2.5. RESTORING AN ARCHIVED PIPELINE EXPERIMENT

You can restore an archived pipeline experiment to the active state.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and has a pipeline server.
- An archived pipeline experiment exists in your project.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Experiments**.
2. On the **Experiments** page, from the **Project** drop-down list, select the project that contains the archived pipeline experiment that you want to restore.
3. Click the **Archive** tab.
4. Click the action menu () beside the pipeline experiment that you want to restore, and then click **Restore**.
5. In the **Restore experiment** dialog, click **Restore**.

Verification

- The restored pipeline experiment is displayed on the **Experiments** tab on the **Experiments** page.

2.6. VIEWING PIPELINE TASK EXECUTIONS

When a pipeline run executes, you can view details of executed tasks in each step in a pipeline run from the OpenShift AI dashboard. A step forms part of a task in a pipeline.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a pipeline server.
- You have imported a pipeline to an active pipeline server.
- You have previously triggered a pipeline run.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Executions**.
2. On the **Executions** page, from the **Project** drop-down list, select the project that contains the experiment for the pipeline task executions that you want to view.

Verification

- On the **Executions** page, you can view the execution details of each pipeline task execution, such as its name, status, unique ID, and execution type. The execution status indicates whether the pipeline task has successfully executed. For further information about the details of the task execution, click the execution name.

2.7. VIEWING PIPELINE ARTIFACTS

After a pipeline run executes, you can view its pipeline artifacts from the OpenShift AI dashboard. Pipeline artifacts can help you to evaluate the performance of your pipeline runs and make it easier to understand your pipeline components. Pipeline artifacts can range from plain text data to detailed, interactive data visualizations.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a pipeline server.
- You have imported a pipeline to an active pipeline server.
- You have previously triggered a pipeline run.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Artifacts**.
2. On the **Artifacts** page, from the **Project** drop-down list, select the project that contains the pipeline experiment for the pipeline artifacts that you want to view.
3. Click an artifact name in the list to view additional information about the artifact, including its original run, original execution, and properties.
4. To view or download the content of an artifact stored in S3-compatible object storage, click the preview icon, the download icon, or the active artifact URI link.
Clicking the preview icon or the URI link for content that your browser can display (such as plain text, HTML, or markdown) opens the artifact in a new browser tab. Clicking the download icon or the URI link for content that your browser cannot display (such as a model file) downloads the

artifact. To download an artifact that is displayed in a browser tab, right-click the content and then click **Save as**.

Verification

- On the **Artifacts** page, you can view the details of each pipeline artifact, such as its name, unique ID, type, and URL.

2.8. COMPARING RUNS IN AN EXPERIMENT

You can compare up to 10 pipeline runs in the same experiment at one time, and view available parameter, scalar metric, confusion matrix, and receiver operating characteristic (ROC) curve data for all selected runs.

To compare runs from different experiments or pipelines, or to view every pipeline run in a project, see [Comparing runs in different experiments](#).

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and has a pipeline server.
- You have imported a pipeline to an active pipeline server.
- You have created at least two pipeline runs.

Procedure

1. In the OpenShift AI dashboard, select **Develop & train** → **Experiments**.
The **Experiments** page opens.
2. From the **Project** drop-down list, select the project that contains the runs that you want to compare.
3. On the **Experiments** tab, in the **Experiment** column, click the experiment that you want to compare runs for. To select runs that are not in an experiment, click **Default**. All runs that are created without specifying an experiment will appear in the **Default** group.
The **Runs** page opens.
4. Select the checkbox next to each run that you want to compare, and then click **Compare runs**.
You can compare a maximum of 10 runs at one time.
The **Compare runs** page opens and displays data for the runs that you selected.
 - a. The **Run list** section displays a list of selected runs. You can filter the list by run name, pipeline version, start date, and status.
 - b. The **Parameters** section displays parameter information for each selected run. Set the **Hide parameters with no differences** switch to **On** to hide parameters that have the same values.
 - c. The **Metrics** section displays scalar metric, confusion matrix, and ROC curve data for all selected runs.
 - i. On the **Scalar metrics** tab, set the **Hide parameters with no differences** switch to **On** to hide parameters that have the same values.

- ii. On the **ROC curve** tab, in the artifacts list, adjust the ROC curve chart by clearing the checkbox next to artifacts that you want to remove from the chart.
5. To select different runs for comparison, click **Manage runs**.
The **Manage runs** dialog opens.
 - a. From the filter drop-down list, select **Run**, **Pipeline version**, **Created after**, or **Status** to filter the run list by each value.
 - b. Clear the checkbox next to each run that you want to remove from your comparison.
 - c. Select the checkbox next to each run that you want to add to your comparison.
6. Click **Update**.

Verification

- The **Compare runs** page opens and displays data for the runs that you selected.

2.9. COMPARING RUNS IN DIFFERENT EXPERIMENTS

You can compare up to 10 pipeline runs from any experiment or pipeline in a project, including runs that do not have a corresponding pipeline, and view available parameter, scalar metric, confusion matrix, and receiver operating characteristic (ROC) curve data for all selected runs.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and has a pipeline server.
- You have imported a pipeline to an active pipeline server.
- You have created at least two pipeline runs.

Procedure

1. In the OpenShift AI dashboard, select **Develop & train** → **Pipelines** → **Runs**.
The **Runs** page opens.
2. From the **Project** drop-down list, select the project that contains the runs that you want to compare.
3. On the **Runs** tab, in the **Run** column, select the checkbox next to each run that you want to compare, and then click **Compare runs**. You can compare a maximum of 10 runs at one time.
The **Compare runs** page opens and displays data for the runs that you selected.
 - a. The **Run list** section displays a list of selected runs. You can filter the list by run name, experiment, pipeline version, start date, and status.
 - b. The **Parameters** section displays parameter information for each selected run. Set the **Hide parameters with no differences** switch to **On** to hide parameters that have the same values.
 - c. The **Metrics** section displays scalar metric, confusion matrix, and ROC curve data for all selected runs.

- i. On the **Scalar metrics** tab, set the **Hide parameters with no differences** switch to **On** to hide parameters that have the same values.
 - ii. On the **ROC curve** tab, in the artifacts list, adjust the ROC curve chart by clearing the checkbox next to artifacts that you want to remove from the chart.
4. To select different runs for comparison, click **Manage runs**.
The **Manage runs** page opens.
 - a. From the filter drop-down list, select **Run**, **Experiment**, **Pipeline version**, **Created after**, or **Status** to filter the run list by each value.
 - b. Clear the checkbox next to each run that you want to remove from your comparison.
 - c. Select the checkbox next to each run that you want to add to your comparison.
5. Click **Update**.

Verification

- The **Compare runs** page opens and displays data for the runs that you selected.

CHAPTER 3. MANAGE PIPELINE RUNS

3.1. PIPELINE RUN TYPES AND LIFECYCLE

A pipeline run is a single execution of an AI pipeline. As data scientist, you can use OpenShift AI to define, manage, and track executions of a pipeline. To view a record of previously executed, scheduled, and archived runs, you must first select the experiment from the **Develop & train → Experiments** page in the OpenShift AI interface. After selecting the experiment, you can access all of its pipeline runs from the **Runs** page.

You can optimize your use of pipeline runs for portability and repeatability by using pipeline experiments. With experiments, you can logically group pipeline runs and try different configurations of your pipelines. You can also clone your pipeline runs to reproduce and scale them, or archive them when you want to retain a record of their execution, but no longer require them. You can delete archived runs that you no longer want to retain, or you can restore them to their former state.

You can execute a run once, that is, immediately after its creation, or on a recurring basis. Recurring runs consist of a copy of a pipeline with all of its parameter values and a run trigger. A run trigger indicates when a recurring run executes. You can define the following run triggers:

- **Periodic:** Used for scheduling runs to execute in intervals
- **Cron:** Used for scheduling runs as a cron job

You can also configure up to 10 instances of the same run to execute concurrently. You can track the progress of a run from the run details page on the OpenShift AI user interface. From here, you can view the graph and output artifacts for the run.

A pipeline run can be in one of the following states:

- **Scheduled:** A pipeline run that is scheduled to execute at least once
- **Active:** A pipeline run that is executing, or stopped
- **Archived:** An archived pipeline run

You can use catch up runs to ensure your pipeline runs do not permanently fall behind schedule when paused. For example, if you re-enable a paused recurring run, the run scheduler backfills each missed run interval. If you disable catch up runs, and you have a scheduled run interval ready to execute, the run scheduler only schedules the run execution for the latest run interval. Catch up runs are enabled by default. However, if your pipeline handles backfill internally, Red Hat recommends that you disable catch up runs to avoid duplicate backfill.

After a pipeline run executes, you can view details of its executed tasks on the **Develop & train → Pipelines → Executions** page, along with its artifacts, on the **Develop & train → Pipelines → Artifacts** page. From the **Executions** page, you can view the execution status of each task, which indicates whether it completed successfully. You can also view further information about each executed task by clicking the execution name in the list. From the **Artifacts** page, you can view the details of each pipeline artifact, such as its name, unique ID, type, and URI. Pipeline artifacts can help you to evaluate the performance of your pipeline runs and make it easier to understand your pipeline components. Pipeline artifacts can range from plain text data to detailed, interactive data visualizations.

You can view further information about each artifact, including its original run and original execution, by clicking the artifact name in the list. You can also view or download the content of artifacts stored in S3-compatible object storage by clicking the preview icon, the download icon, or the active artifact URI link.

Clicking the preview icon or the URI link for content that your browser can display (such as plain text, HTML, or markdown) opens the artifact in a new browser tab. Clicking the download icon or the URI link for content that your browser cannot display (such as a model file) downloads the artifact. To download an artifact that is displayed in a browser tab, right-click the content and then click **Save as**.



NOTE

Artifacts that are not stored in S3-compatible object storage are not available to download and do not display an active URI link.

You can review and analyze logs for each step in an active pipeline run. With the log viewer, you can search for specific log messages, view the log for each step, and download the step logs to your local machine.

3.2. DATA STORAGE WITH PIPELINES

When you run an AI pipeline, OpenShift AI stores the pipeline YAML configuration file and resulting pipeline run artifacts in the **root** directory of your storage bucket. The directories that contain pipeline run artifacts can differ depending on where you executed the pipeline run from. See the following table for further information:

Table 3.1. Pipeline configuration file and artifacts storage locations

Pipeline run source	Pipeline storage directory	Run artifacts storage directory
OpenShift AI dashboard	/pipelines/<pipeline_version_id> Example: /pipelines/1d01c4eb-d2ab-4916-9935-a73a5580f1fb	/<pipeline_name>/<pipeline_run_id> Example: iris-training-pipeline/2g48k8pw-a8ib-4884-9145-h41j7599h3ds
JupyterLab Elyra extension	/pipelines/<pipeline_version_id>	/<pipeline_name_timestamp> Example: /hello-generic-world-0523161704 With the JupyterLab Elyra extension, you can also set an object storage path prefix Example: /iris-project/hello-generic-world-0523161704

If you want to use an existing artifact that was not generated by a task in a pipeline, you can use the [kfp.dsl.importer component](#) to import the artifact from its URI. You can only import these artifacts to the S3-compatible object storage bucket that you define in the **Bucket** field in your pipeline server configuration. For more information about the **kfp.dsl.importer** component, see [Special Case: Importer Components](#).

3.3. PIPELINE RUN WORKSPACES

The workspace feature in AI pipelines provides standardized ephemeral shared storage that exists only for the duration of a pipeline run. This storage lets pipeline components exchange large intermediate data without repeatedly uploading to or downloading from object storage, improving performance and reducing object storage costs.

A workspace is a standardized directory structure backed by a Persistent Volume Claim (PVC) that is dynamically created for each pipeline run based on the **DataSciencePipelinesApplication** (DSPA) workspace configuration. When the pipeline definition uses a workspace, the workspace PVC is automatically mounted into each pipeline task pod that requires it. At runtime, the mount path is resolved by using the **dsl.WORKSPACE_PATH_PLACEHOLDER** variable as an input parameter to components.

Pipelines opt into workspace support at compile time. When the pipeline definition includes a workspace configuration, the pipeline compiler annotates the run so that the DSPA automatically creates the workspace PVC and mounts the volume into each task pod that needs it.

3.3.1. Configure default workspace PVC settings in DSPA

You can configure the default values for workspace Persistent Volume Claims (PVCs) in your **DataSciencePipelinesApplication** (DSPA). This allows you to specify the storage class and access mode for workspace volumes used by pipeline runs. You must provide the workspace storage size when you define the pipeline.

Prerequisites

- You have logged in to OpenShift AI.
- You have the required roles and permissions to edit DSPAs in your project.
- You have configured a pipeline server in your project.

Procedure

1. Log in to the OpenShift web console.
2. Navigate to your project namespace.
3. Click **Search** → **Resources** and select **DataSciencePipelinesApplication** from the resource list.
4. Select your DSPA instance and click the **YAML** tab to edit the configuration.
5. Add or update the workspace configuration under the **spec.apiServer.workspace** field:

```
apiVersion: datasciencepipelinesapplications.opendatahub.io/v1
kind: DataSciencePipelinesApplication
metadata:
  name: sample-dspa
  namespace: my-namespace
spec:
  dspVersion: v2
  apiServer:
    deploy: true
  workspace:
    volumeClaimTemplateSpec:
      accessModes:
        - ReadWriteOnce
```

```
storageClassName: standard-csi
# ... other DSPA configuration
```

- **apiServer.workspace.volumeClaimTemplateSpec** configures the PVC template for workspace volumes.
- **accessModes** specifies the access mode for workspace PVCs. Valid values are **ReadWriteOnce**, **ReadWriteMany**, or **ReadOnlyMany**.
- **storageClassName** specifies the storage class used for workspace PVCs.
- Additional PVC fields supported by Kubernetes, such as **resources** or **persistentVolumeReclaimPolicy**, can also be configured if needed.

6. Click **Save** to apply the changes.

Verification

1. Run the following sample pipeline. It writes a file into the workspace by using **dsl.WORKSPACE_PATH_PLACEHOLDER** and then reads the same file in a later task:

```
from kfp import dsl

@dsl.component
def write_to_workspace(workspace_path: str) -> str:
    """Write a file to the workspace."""
    import os

    file_path = os.path.join(workspace_path, "data", "test_file.txt")
    os.makedirs(os.path.dirname(file_path), exist_ok=True)

    with open(file_path, "w") as f:
        f.write("Hello from workspace!")

    print(f"Wrote file to: {file_path}")
    return file_path

@dsl.component
def read_from_workspace(file_path: str) -> str:
    """Read a file from the workspace using the provided file path."""
    import os

    if os.path.exists(file_path):
        with open(file_path, "r") as f:
            content = f.read()
        print(f"Read content from: {file_path}")
        print(f"Content: {content}")
        assert content == "Hello from workspace!"
        return content

    print(f"File not found at: {file_path}")
    return "File not found"

@dsl.pipeline(
    name="pipeline-with-workspace",
    description="A pipeline that demonstrates workspace functionality",
```

```

    pipeline_config=dsl.PipelineConfig(
        workspace=dsl.WorkspaceConfig(
            size='100Mi',
        ),
    ),
)
def pipeline_with_workspace() -> str:
    """A pipeline using workspace functionality with write and read components."""

    write_task = write_to_workspace(
        workspace_path=dsl.WORKSPACE_PATH_PLACEHOLDER,
    )

    read_task = read_from_workspace(
        file_path=write_task.output,
    )

    return read_task.output

```

2. When the pipeline completes, review the task logs to verify that both write and read operations succeeded.

Additional resources

- [Special Case: Importer Components](#)

3.3.2. Add external artifacts to pipeline run workspaces

When using **dsl.importer** with external artifacts such as Modelcar images stored in an Open Container Initiative (OCI) registry, you cannot download them directly into the workspace. To make the artifact available to subsequent tasks without additional downloads, copy the artifact into the workspace volume in a separate pipeline step.

Prerequisites

- You have configured a pipeline server in your project with workspace support enabled.
- You can reach the external location that stores your artifacts.
- Your pipeline server has the credentials and network access required to pull from that location.

Procedure

To use an external artifact in your pipeline, define a component that copies the artifact to the workspace, and then call this component in your pipeline.

1. Define a component that copies the imported artifact to the workspace:

```

from kfp import dsl

@dsl.component()
def copy_model_to_workspace(input_model: dsl.Input[dsl.Model], workspace_path: str):
    import os
    import shutil
    shutil.copytree(input_model.path, os.path.join(workspace_path, "my-model"))

```

2. Define your pipeline with workspace configuration:

```
@dsl.pipeline(
    name="modelcar-to-workspace",
    pipeline_config=dsl.PipelineConfig(
        workspace=dsl.WorkspaceConfig(size='20Gi'),
    ),
)
def modelcar_to_workspace_pipeline(
    oci_uri: str = "oci://registry.redhat.io/rhelai1/modelcar-granite-3.1-8b-lab-v2.2:latest",
):
    # Import the model from the OCI registry
    model_source = dsl.importer(
        artifact_uri=oci_uri,
        artifact_class=dsl.Model,
    )

    # Copy the model to the workspace
    copy_model_to_workspace(
        input_model=model_source.output,
        workspace_path=dsl.WORKSPACE_PATH_PLACEHOLDER,
    )
```

- The **pipeline_config** parameter with the workspace size is required to enable workspace support for the pipeline run.

3. Compile and upload your pipeline to OpenShift AI.

Verification

1. Navigate to the **Run details** page in the OpenShift AI dashboard.
2. Verify that the **copy_model_to_workspace** task completes successfully.
3. Check the logs of subsequent tasks to confirm they can access the model files in the workspace (for example, **/kfp-workspace/my-model**).



NOTE

The model files are copied to a subdirectory in the workspace (in this example, **my-model**). Subsequent pipeline tasks can access these files at the workspace mount path resolved at runtime.

Additional resources

- [Special Case: Importer Components](#)

3.4. VIEW ACTIVE PIPELINE RUNS

You can view a list of pipeline runs that were previously executed in a pipeline experiment. From this list, you can view details relating to your pipeline runs, such as the pipeline version that the run belongs to, along with the run status, duration, and execution start time.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and has a pipeline server.
- You have imported a pipeline to an active pipeline server.
- You have previously executed a pipeline run that is available.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Experiments**.
2. On the **Experiments** page, from the **Project** drop-down list, select the project that contains the pipeline experiment for the active pipeline runs that you want to view.
3. From the list of experiments, click the experiment that contains the active pipeline runs that you want to view.
The **Runs** page opens.

After a run has completed its execution, the run status is displayed in the **Status** column of the **Runs** tab, indicating whether the run succeeded or failed.

Verification

- A list of active runs is displayed on the **Runs** tab on the **Runs** page for the pipeline experiment.

3.5. EXECUTE A PIPELINE RUN

By default, a pipeline run executes once immediately after it is created.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a configured pipeline server.
- You have imported a pipeline to an active pipeline server.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Experiments**.
2. On the **Experiments** page, from the **Project** drop-down list, select the project that contains the pipeline experiment that you want to create a run for.
3. From the list of pipeline experiments, click the experiment that you want to create a run for.
4. Click **Create run**.
5. On the **Create run** page, configure the run:
 - a. From the **Experiment** list, select the pipeline experiment that you want to create a run for. Alternatively, to create a new pipeline experiment, click **Create new experiment**, and then complete the relevant fields in the **Create experiment** dialog.
 - b. In the **Name** field, enter a name for the run, up to 255 characters.

- c. In the **Description** field, enter a description for the run, up to 255 characters.
- d. From the **Pipeline** list, select the pipeline that you want to create a run for. Alternatively, to create a new pipeline, click **Create new pipeline**, and then complete the relevant fields in the **Import pipeline** dialog.
- e. From the **Pipeline version** list, select the pipeline version to create a run for. Alternatively, to upload a new version, click **Upload new version**, and then complete the relevant fields in the **Upload new version** dialog.
- f. Configure the input parameters for the run by selecting the parameters from the list.
- g. Click **Create run**.
The details page for the run opens.

Verification

- The pipeline run that you created is displayed on the **Runs** tab on the **Runs** page for the pipeline experiment.

3.6. STOP AN ACTIVE PIPELINE RUN

If you no longer require an active pipeline run to continue executing in a pipeline experiment, you can stop the run before its defined end date.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- There is a previously created project available that contains a pipeline server.
- You have imported a pipeline to an active pipeline server.
- An active pipeline run is currently executing.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Experiments**.
2. On the **Experiments** page, from the **Project** drop-down list, select the project that contains the pipeline experiment for the active run that you want to stop.
3. From the list of pipeline experiments, click the pipeline experiment that contains the run that you want to stop.
4. On the **Runs** tab, click the action menu () beside the active run that you want to stop, and then click **Stop**.
There might be a short delay while the run stops.

Verification

- The **Failed** status icon () is displayed in the **Status** column of the stopped run.

3.7. DUPLICATE AN ACTIVE PIPELINE RUN

To make it easier to quickly execute pipeline runs with the same configuration in a pipeline experiment, you can duplicate them.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a configured pipeline server.
- You have imported a pipeline to an active pipeline server.
- An active run is available to duplicate on the **Active** tab on the **Runs** page.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Experiments**.
2. On the **Experiments** page, from the **Project** drop-down list, select the project that contains the pipeline experiment for the pipeline run that you want to duplicate.
3. From the list of pipeline experiments, click the experiment that contains the pipeline run that you want to duplicate.
4. Click the action menu () beside the relevant active run, and then click **Duplicate**.
5. On the **Duplicate run** page, configure the duplicate run:
 - a. From the **Experiment** list, select a pipeline experiment to contain the duplicate run. Alternatively, to create a new pipeline experiment, click **Create new experiment**, and then complete the relevant fields in the **Create experiment** dialog.
 - b. In the **Name** field, enter a name for the duplicate run.
 - c. In the **Description** field, enter a description for the duplicate run.
 - d. From the **Pipeline** list, select a pipeline to contain the duplicate run. Alternatively, to create a new pipeline, click **Create new pipeline**, and then complete the relevant fields in the **Import pipeline** dialog.
 - e. From the **Pipeline version** list, select a pipeline version to contain the duplicate run. Alternatively, to upload a new version, click **Upload new version**, and then complete the relevant fields in the **Upload new version** dialog.
 - f. In the **Parameters** section, configure input parameters for the duplicate run by selecting parameters from the list.
 - g. Click **Create run**.
The details page for the run opens.

Verification

- The duplicate pipeline run is displayed on the **Runs** tab on the **Runs** page for the pipeline experiment.

3.8. VIEW SCHEDULED PIPELINE RUNS

You can view a list of pipeline runs that are scheduled for execution in a pipeline experiment. From this list, you can view details relating to your pipeline runs, such as the pipeline version that the run belongs to. You can also view the run status, execution frequency, and schedule.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a pipeline server.
- You have imported a pipeline to an active pipeline server.
- You have scheduled a pipeline run that is available to view.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Experiments**.
2. On the **Experiments** page, from the **Project** drop-down list, select the project that contains the pipeline experiment for the scheduled pipeline runs that you want to view.
3. From the list of pipeline experiments, click the experiment that contains the pipeline runs that you want to view.
4. On the **Runs** page, click the **Schedules** tab.
After a run is scheduled, the **Status** column indicates whether the run is ready or unavailable for execution. To change its execution availability, set the **Status** switch to **On** or **Off**. Alternatively, you can change its execution availability from the details page for the scheduled run by clicking the **Actions** drop-down menu, and then selecting **Enable** or **Disable**.

Verification

- A list of scheduled runs is displayed on the **Schedules** tab on the **Runs** page for the pipeline experiment.

3.9. PIPELINE RUN SCHEDULING WITH CRON JOBS

You can use a cron job to schedule a pipeline run to execute at a specific time. Cron jobs are useful for creating periodic and recurring tasks, and can also schedule individual tasks for a specific time, such as if you want to schedule a run for a low activity period. To successfully execute runs in OpenShift AI, you must use the supported format. See [Cron Expression Format](#) for more information.

The following examples show the correct format:

Run occurrence	Cron format
Every five minutes	@every 5m
Every 10 minutes	0 */10 * * * *
Daily at 16:16 UTC	0 16 16 * * *
Daily every quarter of the hour	0 0,15,30,45 * * * *

Run occurrence	Cron format
On Monday and Tuesday at 15:40 UTC	0 40 15 * * MON,TUE

Additional resources

- [Cron Expression Format](#)

3.10. SCHEDULE A PIPELINE RUN

To repeatedly run a pipeline, you can create a scheduled pipeline run.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a configured pipeline server.
- You have imported a pipeline to an active pipeline server.

Procedure

- To go to the **Schedules** tab for a run, perform one of the following sets of actions:
 - **To select a run from an experiment**
 - From the OpenShift AI dashboard, click **Develop & train** → **Experiments**.
 - On the **Experiments** page, from the **Project** drop-down list, select the project that contains the pipeline experiment for the run that you want to schedule.
 - From the list of pipeline experiments, click the experiment that contains the run that you want to schedule.
 - Click the **Schedules** tab.
 - **To select any run**
 - From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Runs**.
 - On the **Runs** page, from the **Project** drop-down list, select the project that contains the run that you want to schedule.
 - Click the **Schedules** tab.
- Click **Create schedule**.
- On the **Create schedule** page, configure the run that you are scheduling:
 - From the **Experiment** list, select the pipeline experiment that you want to contain the scheduled run. Alternatively, to create a new pipeline experiment, click **Create new experiment**, and then complete the relevant fields in the **Create experiment** dialog.
 - In the **Name** field, enter a name for the run.

- c. In the **Description** field, enter a description for the run.
- d. From the **Trigger type** list, select one of the following options:
 - Select **Periodic** to specify an execution frequency. In the **Run every** field, enter a number and select an execution frequency from the list.
 - Select **Cron** to specify the execution schedule in **cron** format in the **Cron string** field.

 This creates a cron job to execute the run. Click the **Copy** button () to copy the cron job schedule to the clipboard. The field furthest to the left represents seconds. For more information about scheduling tasks using the supported **cron** format, see [Cron Expression Format](#).
- e. In the **Maximum concurrent runs** field, specify the number of runs that can execute concurrently, from a range of one to ten.
- f. For **Start date**, specify a start date for the run. Select a start date using the calendar, and the start time from the list of times.
- g. For **End date**, specify an end date for the run. Select an end date using the calendar, and the end time from the list of times.
- h. For **Catch up**, enable or disable catch up runs. You can use catch up runs to ensure your pipeline runs do not permanently fall behind schedule when they are paused. For example, if you re-enable a paused recurring run, the run scheduler backfills each missed run interval.
- i. From the **Pipeline** list, select the pipeline that you want to create a run for. Alternatively, to create a new pipeline, click **Create new pipeline**, and then complete the relevant fields in the **Import pipeline** dialog.
- j. For **Pipeline version**, select one of the following options:
 - Select **Always use the latest pipeline version** so that each recurring run automatically uses the most recent pipeline version.
 - Select **Use fixed version**, then select a specific pipeline version for all recurring runs. Alternatively, to upload a new version, click **Upload new version**, and then complete the relevant fields in the **Upload new version** dialog.
- k. Configure the input parameters for the run by selecting the parameters from the list.
- l. Click **Create schedule**.

Verification

- The pipeline run that you scheduled is displayed on the **Schedules** tab on the **Runs** page for the pipeline experiment.

3.11. DUPLICATE A SCHEDULED PIPELINE RUN

To make it easier to schedule runs to execute as part of your pipeline experiment, you can duplicate existing scheduled runs.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a configured pipeline server.
- You have imported a pipeline to an active pipeline server.
- A scheduled run is available to duplicate on the **Schedules** tab on the **Runs** page.

Procedure

- To go to the **Schedules** tab for a run, perform one of the following sets of actions:
 - **To select a run from an experiment**
 - From the OpenShift AI dashboard, click **Develop & train** → **Experiments**.
 - On the **Experiments** page, from the **Project** drop-down list, select the project that contains the pipeline experiment for the run that you want to duplicate.
 - From the list of pipeline experiments, click the experiment that contains the run that you want to duplicate.
 - Click the **Schedules** tab.
 - **To select any run**
 - From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Runs**.
 - On the **Runs** page, from the **Project** drop-down list, select the project that contains the run that you want to schedule.
 - Click the **Schedules** tab.
- Click the action menu () beside the run that you want to duplicate, and then click **Duplicate**.
- On the **Duplicate schedule** page, configure the duplicate run:
 - From the **Experiment** list, select a pipeline experiment to contain the duplicate run. Alternatively, to create a new pipeline experiment, click **Create new experiment**, and then complete the relevant fields in the **Create experiment** dialog.
 - In the **Name** field, enter a name for the duplicate run.
 - In the **Description** field, enter a description for the duplicate run.
 - From the **Trigger type** list, select one of the following options:
 - Select **Periodic** to specify an execution frequency. In the **Run every** field, enter a number, and select an execution frequency from the list.
 - Select **Cron** to specify the execution schedule in **cron** format in the **Cron string** field.

This creates a cron job to execute the run. Click the **Copy** button () to copy the cron job schedule to the clipboard. The field furthest to the left represents seconds. For more information about scheduling tasks using the supported **cron** format, see [Cron Expression Format](#).

- e. For **Maximum concurrent runs**, specify the number of runs that can execute concurrently, from a range of one to ten.
- f. For **Start date**, specify a start date for the duplicate run. Select a start date using the calendar, and the start time from the list of times.
- g. For **End date**, specify an end date for the duplicate run. Select an end date using the calendar, and the end time from the list of times.
- h. For **Catch up**, enable or disable catch up runs. You can use catch up runs to ensure your pipeline runs do not permanently fall behind schedule when they are paused. For example, if you re-enable a paused recurring run, the run scheduler backfills each missed run interval.
- i. From the **Pipeline** list, select the pipeline that you want to create a duplicate run for. Alternatively, to create a new pipeline, click **Create new pipeline**, and then complete the relevant fields in the **Import pipeline** dialog.
- j. For **Pipeline version**, select one of the following options:
 - Select **Always use the latest pipeline version** so that each recurring run automatically uses the most recent pipeline version.
 - Select **Use fixed version**, then select a specific pipeline version for all recurring runs. Alternatively, to upload a new version, click **Upload new version**, and then complete the relevant fields in the **Upload new version** dialog.
- k. Configure input parameters for the run by selecting parameters from the list.
- l. Click **Schedule run**.

Verification

- The pipeline run that you duplicated is displayed on the **Schedules** tab on the **Runs** page for the pipeline experiment.

3.12. DELETE A SCHEDULED PIPELINE RUN

To discard pipeline runs that you previously scheduled, but no longer require, you can delete them so that they do not appear on the **Schedules** page.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a configured pipeline server.
- You have imported a pipeline to an active pipeline server.
- You have previously scheduled a run that is available to delete.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Experiments**.
2. On the **Experiments** page, from the **Project** drop-down list, select the project that contains the pipeline experiment for the scheduled pipeline run that you want to delete.

3. From the list of pipeline experiments, click the experiment that contains the scheduled pipeline run that you want to delete.
4. On the **Runs** page, click the **Schedules** tab.
5. Click the action menu () beside the scheduled pipeline run that you want to delete, and then click **Delete**.
6. In the **Delete schedule** dialog, enter the run name in the text field to confirm that you intend to delete it.
7. Click **Delete**.

Verification

- The run that you deleted is no longer displayed on the **Schedules** tab for the pipeline experiment.

3.13. VIEW PIPELINE RUN DETAILS

To gain a clearer understanding of your pipeline runs, you can view the details of a previously triggered pipeline run, such as its graph, execution details, and run output.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a pipeline server.
- You have imported a pipeline to an active pipeline server.
- You have previously triggered a pipeline run.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Runs**.
2. On the **Runs** page, from the **Project** drop-down list, select the project that you want to view the details of a pipeline run for.
3. On the **Runs** page, click the name of the run that you want to view the details of. The details page for the run opens.

Verification

- On the run details page, you can view the run graph, execution details, input parameters, step logs, and run output.

3.14. VIEW ARCHIVED PIPELINE RUNS

You can view a list of pipeline runs that you have archived. You can view details for your archived pipeline runs, such as the pipeline version, run status, duration, and execution start date.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and has a pipeline server.
- You have imported a pipeline to an active pipeline server.
- An archived pipeline run exists.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Experiments**.
2. On the **Experiments** page, from the **Project** drop-down list, select the project that contains the pipeline experiment for the archived pipeline runs that you want to view.
3. From the list of pipeline experiments, click the experiment that contains the archived pipeline runs that you want to view.
4. On the **Runs** page, click the **Archive** tab.

Verification

- A list of archived runs is displayed on the **Archive** tab on the **Runs** page for the pipeline experiment.

3.15. ARCHIVE A PIPELINE RUN

You can retain records of your pipeline runs by archiving them. If required, you can restore runs from your archive to reuse, or delete runs that are no longer required.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and has a pipeline server.
- You have imported a pipeline to an active pipeline server.
- You have previously executed a pipeline run that is available.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Experiments**.
2. On the **Experiments** page, from the **Project** drop-down list, select the project that contains the pipeline experiment for the run that you want to archive.
3. From the list of pipeline experiments, click the experiment that contains the pipeline run that you want to archive.
The **Runs** page opens.
4. On the **Runs** tab, click the action menu () beside the pipeline run that you want to archive, and then click **Archive**.
5. In the **Archiving run** dialog, enter the run name in the text field to confirm that you intend to archive it.

6. Click **Archive**.

Verification

- The archived run does not appear on the **Runs** tab, and instead is displayed on the **Archive** tab on the **Runs** page for the pipeline experiment.

3.16. RESTORE AN ARCHIVED PIPELINE RUN

You can restore an archived run to the active state.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and has a pipeline server.
- You have imported a pipeline to an active pipeline server.
- An archived run exists in your project.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train → Experiments**.
2. On the **Experiments** page, from the **Project** drop-down list, select the project that contains the pipeline experiment that you want to restore.
3. From the list of pipeline experiments, click the experiment that contains the archived pipeline run that you want to restore.
4. On the **Runs** page, click the **Archive** tab.
5. Click the action menu () beside the pipeline run that you want to restore, and then click **Restore**.
6. In the **Restore run?** dialog, click **Restore**.

Verification

- The restored run is displayed on the **Runs** tab on the **Runs** page for the pipeline experiment.

3.17. DELETE AN ARCHIVED PIPELINE RUN

You can delete pipeline runs from the OpenShift AI run archive.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and has a pipeline server.
- You have imported a pipeline to an active pipeline server.
- You have previously archived a pipeline run.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Runs**.
2. On the **Runs** page, from the **Project** drop-down list, select the project that you want to delete an archived pipeline run from.
3. Click the **Archive** tab.
4. Click the action menu () beside the pipeline run that you want to delete, and then click **Delete**.
5. In the **Delete run?** dialog, enter the run name in the text field to confirm that you intend to delete it.
6. Click **Delete**.

Verification

- The archived run that you deleted is no longer displayed on the **Archive** tab on the **Runs** page.

3.18. DUPLICATE AN ARCHIVED PIPELINE RUN

To make it easier to reproduce runs with the same configuration as runs in your archive, you can duplicate them.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a configured pipeline server.
- You have imported a pipeline to an active pipeline server.
- An archived run is available to duplicate on the **Archived** tab on the **Runs** page.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Runs**.
2. On the **Runs** page, from the **Project** drop-down list, select the project that you want to duplicate a pipeline run for.
3. On the **Runs** page, click the **Archive** tab.
4. Click the action menu () beside the pipeline run that you want to duplicate, and then click **Duplicate**.
5. On the **Duplicate run** page, configure the duplicate run:
 - a. From the **Experiment** list, select a pipeline experiment to contain the duplicate run. Alternatively, to create a new pipeline experiment, click **Create new experiment**, and then complete the relevant fields in the **Create experiment** dialog.
 - b. In the **Name** field, enter a name for the duplicate run.
 - c. In the **Description** field, enter a description for the duplicate run.

- d. From the **Pipeline** list, select a pipeline to contain the duplicate run. Alternatively, to create a new pipeline, click **Create new pipeline**, and then complete the relevant fields in the **Import pipeline** dialog.
- e. From the **Pipeline version** list, select a pipeline version to contain the duplicate run. Alternatively, to upload a new version, click **Upload new version**, and then complete the relevant fields in the **Upload new version** dialog.
- f. In the **Parameters** section, configure input parameters for the duplicate run by selecting parameters from the list.
- g. Click **Create run**.
The details page for the run opens.

Verification

- The duplicate pipeline run is displayed on the **Runs** tab on the **Runs** page for the pipeline experiment.

CHAPTER 4. TRACK PIPELINE EXPERIMENTS WITH MLFLOW

You can use MLflow experiment tracking with OpenShift AI pipelines to automatically log pipeline run results, parameters, and metrics to an MLflow server. You can then compare training runs across pipelines, analyze parameters, and find optimal results without modifying your pipeline code.

4.1. MLFLOW EXPERIMENT TRACKING FOR PIPELINE RUNS

When MLflow integration is enabled for your pipeline server, OpenShift AI automatically creates a parent MLflow run for each pipeline run and nested runs for each task, grouped under an MLflow experiment. This integration requires no changes to your pipeline code. New runs of existing pipelines are automatically uploaded as MLflow runs under a specified or default experiment when the integration is active.

4.1.1. MLflow run mapping structure

Each pipeline run creates the following structure in MLflow:

Parent MLflow run

OpenShift AI creates one parent MLflow run for each pipeline run. The parent run is tagged with the pipeline run URL for cross-referencing.

Nested MLflow runs

Each task in the pipeline creates a nested MLflow run under the parent. If a task runs in a parallel loop, each iteration creates its own nested run.

Through this hierarchy, you can view the overall pipeline result at the parent level and inspect individual task results through the nested runs.

For more complex pipeline structures, the MLflow run hierarchy reflects the pipeline topology:

Nested pipelines

When a parent pipeline calls a child pipeline as a task, the child pipeline is represented as a separate, intermediary MLflow run between the parent pipeline run and the child pipeline's individual task runs.

Looped tasks

When a pipeline uses parallel loops, the loop creates a nested MLflow run under the parent, and each iteration of the loop creates its own nested run under the loop run.

4.1.2. Automatically logged data

The integration automatically logs the following data to MLflow for each task:

- Input parameters
- Scalar output metrics
- Run status: completed or failed

Artifacts such as model files and datasets are not synchronized to MLflow. They remain in the object storage configured for your pipeline server.

4.1.3. Default experiment behavior

When you create a pipeline run, you can optionally specify an MLflow experiment name. If you do not specify an experiment, the run is tracked under the MLflow "Default" experiment. If the specified experiment does not exist, OpenShift AI creates it automatically.

4.1.4. Graceful degradation

If the MLflow server is unreachable when a pipeline run executes, the pipeline run still completes successfully. The MLflow tracking data is incomplete, but pipeline execution is not affected.

4.1.5. Direct SDK access from pipeline tasks

When an administrator sets **injectUserEnvVars** to **true** in the **DataSciencePipelinesApplication** custom resource, or selects the **Allow pipeline tasks to log directly to MLflow** checkbox in the pipeline server advanced settings, OpenShift AI sets the following environment variables on each task container:

- **MLFLOW_TRACKING_URI**: the MLflow server endpoint
- **MLFLOW_RUN_ID**: the nested MLflow run ID for the current task
- **MLFLOW_EXPERIMENT_ID**: the active MLflow experiment
- **MLFLOW_WORKSPACE**: the MLflow workspace mapped to the project namespace

These variables allow your pipeline code to call the MLflow Python SDK directly, for example to log additional custom metrics with **mlflow.log_metric()**, without configuring the connection or managing run IDs. Authentication is pre-configured automatically.



NOTE

The automatic logging performed by the platform and any direct SDK calls from your pipeline code both write to the same nested MLflow run.

4.1.6. MLflow run view compared to the pipeline run view

The MLflow experiments page provides capabilities beyond the standard pipeline run view:

- Hierarchical navigation between parent and nested runs
- Side-by-side comparison of metrics and parameters across runs
- Interactive visualizations such as parallel coordinates plots, scatter plots, and contour plots

Additional resources

- [Compare MLflow runs](#)

4.2. ENABLE MLFLOW INTEGRATION FOR PIPELINE RUNS

As an administrator, you can enable MLflow experiment tracking for pipeline runs by configuring the pipeline server settings in the **DataSciencePipelinesApplication** (DSPA) custom resource (CR) directly, or by using the OpenShift AI dashboard. When enabled, all pipeline runs in the project are automatically tracked in MLflow.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- The MLflow Operator component is set to **Managed** in the **DataScienceCluster** resource.
- An **MLflow** custom resource has been created.
- You have cluster administrator privileges or you are a project administrator for the project that contains the pipeline server.

Procedure

1. Optional: To enable or disable MLflow integration by editing the CR, edit the **DataSciencePipelinesApplication** (DSPA) custom resource directly. To use the dashboard, continue to the next step.

```
apiVersion: datasciencepipelinesapplications.opendatahub.io/v1alpha1
kind: DataSciencePipelinesApplication
metadata:
  name: dspa
spec:
  mlflow:
    integrationMode: AUTODETECT
    injectUserEnvVars: true
```

where:

integrationMode

Specifies the MLflow integration mode. Set to **AUTODETECT** to enable automatic detection of the MLflow instance in the project. Set to **DISABLED** to turn off MLflow integration. The default value is **AUTODETECT**.

injectUserEnvVars

Specifies whether to inject MLflow environment variables into pipeline task containers. Set to **true** to allow pipeline tasks to log directly to MLflow by using the MLflow Python SDK. The default value is **false**.

2. From the OpenShift AI dashboard, click **Develop & train → Pipelines**.
3. Select the project that contains the pipeline server from the **Project** list.
4. Click **Manage pipeline server**.
If no pipeline server is configured, click **Configure pipeline server** instead.
5. Click **Advanced settings**.
6. In the **MLflow** section, configure the following options:
 - a. Select the **Automatically detect and connect to MLflow** checkbox.
This setting enables the pipeline server to detect and connect to the MLflow instance in your project. When enabled, all pipeline runs are automatically tracked in MLflow experiments.
 - b. Optional: Select the **Allow pipeline tasks to log directly to MLflow** checkbox.

This setting injects MLflow environment variables such as **MLFLOW_TRACKING_URI** and **MLFLOW_RUN_ID** into each pipeline task container. Enable this option if your pipeline code uses the MLflow Python SDK to log custom metrics or parameters.

When this checkbox is enabled, the create run form displays code snippets that show how to log directly to MLflow from pipeline tasks. When this checkbox is cleared, MLflow environment variables are not injected into task containers and the integrated MLflow Python SDK is unavailable within pipeline tasks. However, platform-level tracking of pipeline runs in MLflow continues if **Automatically detect and connect to MLflow** remains enabled.

7. Click **Update** to apply the changes.
8. Optional: To disable MLflow integration for all pipeline runs in the project, click **Pipeline server actions → Manage pipeline server configuration** and clear the **Automatically detect and connect to MLflow** checkbox.
Alternatively, set the **integrationMode** field to **DISABLED** in the **DataSciencePipelinesApplication** custom resource.

Verification

- In the OpenShift AI dashboard, navigate to **Develop & train → Pipelines** and verify that the **MLflow** section is visible in the pipeline interface.
- Create a pipeline run and verify that an MLflow experiment is created in the **Develop & train → Experiments (MLflow)** page.

Additional resources

- [MLflow experiment tracking for pipeline runs](#)
- [Track experiments with MLflow in workbenches](#)

4.3. VIEW MLFLOW EXPERIMENTS FOR PIPELINE RUNS

After MLflow integration is enabled for your pipeline server, you can view MLflow experiment data for your pipeline runs directly from the OpenShift AI dashboard. Each pipeline run is automatically tracked as an MLflow experiment, and you can navigate between the pipeline run view and the MLflow experiment view.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- An administrator has enabled MLflow integration for the pipeline server in your project. For more information, see [Enable MLflow integration for pipeline runs](#).
- You have created and run at least one pipeline.

Procedure

1. When creating a pipeline run, configure experiment tracking for the run in the **MLflow integration** section:
 - a. Verify that the **Enable MLflow experiment tracking** checkbox is selected.

This checkbox is selected by default when MLflow integration is enabled for the pipeline server. Clear this checkbox to disable MLflow tracking for this specific run.

- b. Select **Choose existing MLflow experiment** and select an experiment from the **MLflow experiment** list, or select **Create new MLflow experiment** and enter a name for the new experiment.
If you do not specify an experiment, the run is tracked under the MLflow "Default" experiment.
 - c. Optional: If an administrator has enabled the **Allow pipeline tasks to log directly to MLflow** option, review the MLflow code snippets in the create run form that show how to use the MLflow Python SDK within pipeline tasks.
2. After the pipeline run completes, click **Develop & train → Pipelines → Runs**.
 3. Select the project that contains the pipeline runs from the **Project** list.
The pipeline runs table displays an **MLflow experiment** column that shows the MLflow experiment associated with each run.
 4. To view the MLflow experiment details for a pipeline run, click the experiment name in the **MLflow experiment** column.
The MLflow experiments page opens, showing the parent MLflow run that corresponds to the pipeline run.
 5. To view individual task results, expand the parent run to see nested runs.
Each nested run corresponds to a task in the pipeline and contains the parameters and metrics that the task logged.
 6. Optional: To compare pipeline runs in MLflow, select the runs that have MLflow tracking enabled in the pipeline runs table and click **Compare runs**.
If all selected runs have MLflow tracking, the MLflow compare runs view opens. If any selected run does not have MLflow tracking, the standard pipeline compare runs view opens instead.

For more information about comparing runs, see [Compare MLflow runs](#).



NOTE

When you compare runs in the MLflow experiments page, the comparison view displays metrics and parameters from the parent runs. Metrics logged within nested runs might not appear in the top-level comparison view. To view nested run metrics, navigate to the individual nested run details.

Verification

- Verify that the pipeline runs table displays the **MLflow experiment** column with experiment names for completed runs.
- Click an experiment name and verify that you are redirected to the MLflow experiments page showing the parent run and its nested task runs.

CHAPTER 5. WORK WITH PIPELINE LOGS

5.1. PIPELINE LOGS

You can review and analyze step logs for each step in a triggered pipeline run.

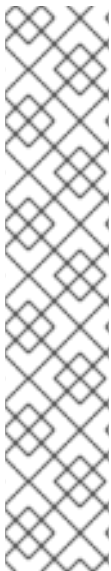
To help you troubleshoot and audit your pipelines, you can review and analyze these step logs by using the log viewer in the OpenShift AI dashboard. From here, you can search for specific log messages, view the log for each step, and download the step logs to your local machine.

If the step log file exceeds its capacity, a warning is displayed above the log viewer stating that the log window displays partial content. Expanding the warning displays further information, such as how the log viewer refreshes every three seconds, and that each step log displays the last 500 lines of log messages received. In addition, you can click **download all step logs** to download all step logs to your local machine.

Each step has a set of container logs. You can view these container logs by selecting a container from the **Steps** list in the log viewer. The **Step-main** container log consists of the log output for the step. The **step-copy-artifact** container log consists of output relating to artifact data sent to s3-compatible storage. If the data transferred between the steps in your pipeline is larger than 3 KB, five container logs are typically available. These logs contain output relating to data transferred between your persistent volume claims (PVCs).

5.2. VIEW PIPELINE STEP LOGS

To help you troubleshoot and audit your pipelines, you can review and analyze the log of each pipeline step using the log viewer. From here, you can search for specific log messages and download the logs for each step in your pipeline. If the pipeline is running, you can also pause and resume the log from the log viewer.



NOTE

Logs are no longer stored in S3-compatible storage for Python scripts which are running in Elyra pipelines. From OpenShift AI version 2.11, you can view these logs in the pipeline step log viewer.

For this change to take effect, you must use the Elyra runtime images provided in workbench images at version 2024.1 or later.

If you have an older workbench image version, update the **Version selection** field to a compatible workbench image version, for example, **2024.1**, as described in [Updating a project workbench](#).

Updating your workbench image version will clear any existing runtime image selections for your pipeline. After you update your workbench version, open your workbench IDE and update the properties of your pipeline to select a runtime image.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a pipeline server.
- You have imported a pipeline to an active pipeline server.

- You have previously triggered a pipeline run.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Runs**.
2. On the **Runs** page, from the **Project** drop-down list, select the project that you want to view pipeline step logs for.
3. On the **Runs** page, click the name of the run that you want to view logs for.
4. On the run details page, on the **Graph** tab, click the pipeline step that you want to view logs for.
5. Click the **Logs** tab.
6. To view the logs of another pipeline step, from the **Steps** list, select the step that you want to view logs for.
7. Analyze the log using the log viewer.
 - To search for a specific log message, enter at least part of the message in the search bar.
 - To view the full log in a separate browser window, click the action menu (⋮) and select **View raw logs**. Alternatively, to expand the size of the log viewer, click the action menu (⋮) and select **Expand**.

Verification

- You can view the logs for each step in your pipeline.

5.3. DOWNLOAD PIPELINE STEP LOGS

Instead of viewing the step logs of a pipeline run using the log viewer on the OpenShift AI dashboard, you can download them for further analysis. You can choose to download the logs belonging to all steps in your pipeline, or you can download the log only for the step log displayed in the log viewer.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have previously created a project that is available and contains a pipeline server.
- You have imported a pipeline to an active pipeline server.
- You have previously triggered a pipeline run.

Procedure

1. From the OpenShift AI dashboard, click **Develop & train** → **Pipelines** → **Runs**.
2. On the **Runs** page, from the **Project** drop-down list, select the project that you want to download logs for.
3. On the **Runs** page, click the name of the run that you want to download logs for.

4. On the run details page, on the **Graph** tab, click the pipeline step that you want to download logs for.
5. Click the **Logs** tab.
6. In the log viewer, click the **Download** button ().
 - a. Select **Download current step log** to download the log for the current pipeline step.
 - b. Select **Download all step logs** to download the logs for all steps in your pipeline run.

Verification

- The step logs download to your browser's default directory for downloaded files.

CHAPTER 6. USE THE AI PIPELINES REST API WITH KUBEFLOW PIPELINES SDK

You can use the Kubeflow pipelines SDK and AI Pipelines REST API to programmatically manage AI pipelines, pipeline versions, experiments, runs, and other resources. This enables automation and integration with external applications.

6.1. AI PIPELINES REST API AND SDK

The AI Pipelines service provides a RESTful API that allows you to programmatically manage pipelines, pipeline versions, experiments, runs, and other resources. You can interact with this API by using the Kubeflow Pipelines SDK, which provides a Python client library with methods for common operations.

6.1.1. REST API overview

The AI Pipelines REST API is based on the Kubeflow pipelines API and provides endpoints for the following operations:

- Managing pipelines and pipeline versions
- Creating and managing experiments
- Executing and monitoring pipeline runs
- Managing artifacts and task executions

The API is exposed through the pipeline server route protected by the OpenShift AI Gateway API authentication layer. You must provide a valid access token when making API requests.

6.1.2. Kubeflow pipelines SDK

The Kubeflow pipelines SDK provides a Python client library that simplifies interaction with the AI Pipelines REST API. The SDK handles request formatting, authentication, and response parsing, allowing you to focus on pipeline orchestration logic.

The SDK is based on Kubeflow Pipelines 2.0 and requires Python 3.11 or later. Before using the SDK, you must install it and authenticate the client with your pipeline server.

6.1.3. Additional resources

- [Installing the Kubeflow Pipelines SDK](#)
- [Authenticating the Kubeflow Pipelines SDK with a pipeline server](#)
- [Kubeflow Pipelines Documentation](#)
- [Kubeflow Pipelines SDK API Reference](#)

6.2. INSTALL THE KUBEFLOW PIPELINES SDK

Install the Kubeflow Pipelines SDK to interact with the AI Pipelines REST API programmatically.

Prerequisites

- You have Python 3.11 or later installed in your environment.

Procedure

1. Open a terminal in your environment.
2. Install the Kubeflow Pipelines SDK:

```
pip install kfp
```



NOTE

The Kubeflow Pipelines SDK version should be compatible with the Kubeflow Pipelines version used by OpenShift AI.

Verification

- Verify that the SDK is installed:

```
python -c "import kfp"
```

The command outputs the installed KFP SDK version.

Next steps

- [Authenticating the Kubeflow Pipelines SDK with a pipeline server](#) .

6.3. WORK WITH PIPELINES BY USING THE KUBEFLOW PIPELINES SDK

Use the Kubeflow Pipelines SDK to upload, list, retrieve, and delete pipelines using the REST API client. Pipelines are the core building blocks that define the workflow of your machine learning tasks.

Prerequisites

- You have installed the Kubeflow Pipelines SDK. For more information, see [Installing the Kubeflow Pipelines SDK](#).
- You have authenticated the SDK client with your pipeline server. For more information, see [Authenticating the Kubeflow Pipelines SDK with a pipeline server](#) .

6.3.1. Uploading a pipeline

Upload pipelines from local files to make them available for execution.

- Use the **upload_pipeline** method:

```
pipeline = client.upload_pipeline(
    pipeline_package_path=<_path/to/pipeline.yaml>,
    pipeline_name='my-pipeline',
    description='My sample pipeline'
)
```

where:

pipeline_package_path

Specifies the local file path to the pipeline definition file in YAML format. Replace `<path/to/pipeline.yaml>` with the local file path.

pipeline_name

Specifies the name for the pipeline. This name must be unique.

description

Specifies an optional description of the pipeline's purpose.

6.3.2. Listing pipelines

- To retrieve a list of all pipelines with support for pagination and filtering, use the **list_pipelines** method:

```
# List all pipelines
pipelines = client.list_pipelines()
print(f"Found {pipelines.total_size} pipelines")

for pipeline in pipelines.pipelines:
    print(f"Pipeline: {pipeline.display_name} (ID: {pipeline.pipeline_id})")
```

- To list pipelines with pagination:

```
pipelines = client.list_pipelines(page_size=10, page_token=None)
```

where:

page_size

Specifies the maximum number of pipelines to return per page.

page_token

Specifies the token for retrieving the next page of results. Use **None** for the first page.

- To list pipelines with filtering:

```
import json
pipeline_filter = json.dumps({
    "predicates": [{
        "operation": "EQUALS",
        "key": "display_name",
        "stringValue": "my-pipeline",
    }]
})
pipelines = client.list_pipelines(filter=pipeline_filter)
```

where:

filter

Specifies a JSON-serialized filter to narrow results. The filter uses predicates with operation types such as **EQUALS**.

6.3.3. Getting pipeline details

Retrieve detailed information about a specific pipeline by ID or name.

- To get a pipeline by ID, use the **get_pipeline** method:

```
pipeline_id = <_pipeline_id_>
pipeline = client.get_pipeline(<_pipeline_id_>)
```

where:

pipeline_id

Specifies the unique identifier of the pipeline to retrieve. Replace `<pipeline_id>` with your actual value, for example, "a1b2c3d4-e5f6-7890-abcd-ef1234567890".

- To get a pipeline by name:

```
pipeline_id = client.get_pipeline_id("my-pipeline")
if pipeline_id:
    pipeline = client.get_pipeline(pipeline_id)
```

The **get_pipeline_id** method returns the ID for a pipeline with the specified name, which you can then use to retrieve the full pipeline details.

6.3.4. Deleting a pipeline

- To remove pipelines from the system by ID, use the **delete_pipeline** method:

```
client.delete_pipeline (<_pipeline_id_>)
```

where:

pipeline_id

Specifies the unique identifier of the pipeline to delete. Replace `<pipeline_id>` with your actual value, for example, "a1b2c3d4-e5f6-7890-abcd-ef1234567890".

- To delete a pipeline by name, first retrieve the pipeline ID:

```
pipeline_id = client.get_pipeline_id('my-pipeline')
if pipeline_id:
    client.delete_pipeline(pipeline_id)
```

6.3.5. Additional resources

- [Working with pipeline versions by using the SDK](#)
- [Kubeflow Pipelines SDK API Reference](#)

6.4. WORK WITH PIPELINE VERSIONS BY USING THE KUBEFLOW PIPELINES SDK

Use the KubeFlow Pipelines SDK to create, list, and retrieve pipeline versions programmatically. Pipeline versions allow you to maintain multiple versions of the same pipeline, enabling version control and tracking of pipeline changes over time.

Prerequisites

- You have installed the KubeFlow Pipelines SDK. For more information, see [Installing the KubeFlow Pipelines SDK](#).
- You have authenticated the SDK client with your pipeline server. For more information, see [Authenticating the KubeFlow Pipelines SDK with a pipeline server](#).

6.4.1. Creating pipeline versions

Create new versions of existing pipelines by uploading updated pipeline definitions from local files.

- Use the **upload_pipeline_version** method:

```
version = client.upload_pipeline_version(
    pipeline_package_path=<_path/to/updated-pipeline.yaml>,
    pipeline_version_name="v2.0",
    pipeline_id="pipeline_id"
)
```

where:

pipeline_package_path

Specifies the local file path to the updated pipeline definition file in YAML format. Replace **<path/to/updated-pipeline.yaml>** with the updated pipeline definition file.

pipeline_version_name

Specifies the name for the pipeline version.

pipeline_id

Specifies the unique identifier of the parent pipeline.

- To upload a pipeline version from a URL, download the file locally, then use the **upload_pipeline_version** method:

```
import urllib.request
urllib.request.urlretrieve("https://github.com/example/pipeline-v2.yaml", 'pipeline-v2.yaml')
version = client.upload_pipeline_version(
    pipeline_package_path='pipeline-v2.yaml',
    pipeline_version_name='v2.1',
    pipeline_id=pipeline_id
)
```

6.4.2. Listing pipeline versions

To retrieve all versions associated with a specific pipeline, use the **list_pipeline_versions** method:

```
versions = client.list_pipeline_versions(<_pipeline_id_>)

for version in versions.pipeline_versions:
```



```
print(f"Version: {version.display_name} (ID: {version.pipeline_version_id})")
```

where:

pipeline_id

Specifies the unique identifier of the pipeline whose versions you want to list. Replace it with your actual pipeline ID.

6.4.3. Getting pipeline version details

To retrieve detailed information about a specific pipeline version, use the **get_pipeline_version** method:



NOTE

Replace `<pipeline_id>` and `<_pipeline_version_id>` with your actual pipeline and version IDs.

```
version = client.get_pipeline_version(
    pipeline_id=<_pipeline_id_>,
    pipeline_version_id=<_pipeline_version_id_>
)
```

where:

pipeline_id

Specifies the unique identifier of the parent pipeline.

pipeline_version_id

Specifies the unique identifier of the pipeline version to retrieve.

6.4.4. Additional resources

- [Overview of pipeline versions](#)
- [Working with pipelines by using the SDK](#)
- [Kubeflow Pipelines SDK API Reference](#)

6.5. WORK WITH EXPERIMENTS BY USING THE KUBEFLOW PIPELINES SDK

Use the Kubeflow Pipelines SDK to create, list, retrieve, archive, and unarchive experiments programmatically. Experiments organize and group related pipeline runs for comparison and tracking.

Prerequisites

- You have installed the Kubeflow Pipelines SDK. For more information, see [Installing the Kubeflow Pipelines SDK](#).
- You have authenticated the SDK client with your pipeline server. For more information, see [Authenticating the Kubeflow Pipelines SDK with a pipeline server](#).

6.5.1. Creating an experiment

Create a new experiment to organize your pipeline runs into logical groups.

Use the **create_experiment** method:

```
experiment = client.create_experiment(  
    name="my-experiment",  
    description="Experiment for testing pipelines",  
)
```

where:

name

Specifies the name for the experiment.

description

Specifies an optional description of the experiment's purpose.

6.5.2. Listing experiments

To retrieve a list of all experiments with support for pagination and sorting, use the **list_experiments** method:

```
# List all experiments  
experiments = client.list_experiments()  
  
for exp in experiments.experiments:  
    print(f"Experiment: {exp.display_name} (ID: {exp.experiment_id})")  
  
# List with pagination and sorting  
experiments = client.list_experiments(  
    page_size=20,  
    sort_by='created_at desc'  
)
```

where:

page_size

Specifies the maximum number of experiments to return per page. Defaults to 10.

sort_by

Specifies the field and order for sorting results. Use **created_at desc** for newest first or **name asc** for alphabetical order.

6.5.3. Getting experiment details

- To retrieve details for a specific experiment by ID, use the **get_experiment** method:

```
experiment = client.get_experiment(  
    experiment_id=<_experiment_id_>  
)
```

where:

experiment_id

Specifies the unique identifier of the experiment to retrieve. Replace **<experiment_id>** with your actual ID, for example: "a1b2c3d4-e5f6-7890-abcd-ef1234567890".

- To retrieve an experiment by name:

```
experiment = client.get_experiment(
    experiment_name="my-experiment"
)
```

where:

experiment_name

Specifies the name of the experiment to retrieve.

6.5.4. Additional resources

- [Overview of pipeline experiments](#)
- [Working with runs by using the SDK](#)
- [Kubeflow Pipelines SDK API Reference](#)

6.6. WORK WITH RUNS BY USING THE SDK

Use the Kubeflow Pipelines SDK to create, monitor, list, and manage pipeline runs programmatically. Runs represent individual executions of pipelines.

Prerequisites

- You have installed the Kubeflow Pipelines SDK. For more information, see [Installing the Kubeflow Pipelines SDK](#).
- You have authenticated the SDK client with your pipeline server.

6.6.1. Creating and submitting runs

Submit pipeline runs with parameters, either from uploaded pipelines or directly from pipeline package files.

- To submit a run with parameters, use the **run_pipeline** method:

```
run = client.run_pipeline(
    experiment_id=experiment.experiment_id,
    job_name="my-pipeline-run",
    pipeline_id=pipeline_id,
    params={
        'learning_rate': 0.01
        'epochs': 50
    }
)
```

where:

experiment_id

Specifies the unique identifier of the experiment to associate with this run.

job_name

Specifies the name for the pipeline run.

pipeline_id

Specifies the unique identifier of the pipeline to execute.

params

Specifies a dictionary of parameter names and values to pass to the pipeline.

- To submit a run with a specific pipeline version:

```
run = client.run_pipeline(  
    experiment_id=experiment.experiment_id,  
    job_name="my-pipeline-run-v2",  
    pipeline_id=pipeline_id,  
    version_id=version_id,  
    params={'param1': 'value1'}  
)
```

where:

version_id

Specifies the unique identifier of the pipeline version to execute.

- To submit a run directly from a pipeline package file, use the **create_run_from_pipeline_package** method:

```
run = client.create_run_from_pipeline_package(  
    pipeline_file=<_path/to/pipeline.yaml>,  
    arguments={'param1': 'value1'},  
    run_name='direct-run',  
    experiment_name='my-experiment'  
)
```

where:

pipeline_file

Specifies the local file path to the pipeline package in YAML format.

arguments

Specifies a dictionary of parameter names and values to pass to the pipeline.

run_name

Specifies the name for the pipeline run.

experiment_name

Specifies the name of the experiment to associate with this run.

6.6.2. Monitoring runs

Track the progress and status of pipeline runs, including waiting for completion.

- Use the **get_run** method to get run details:

```
run_detail = client.get_run(run.run_id)
print(f"Run status: {run_detail.state}")
```

where:

run_id

Specifies the unique identifier of the run to retrieve.

- To wait for a run to complete, use the **wait_for_run_completion** method:

```
client.wait_for_run_completion(run.run_id, timeout=3600) # 1 hour timeout
```

- To get the current run status:

```
run_detail = client.get_run(run.run_id)
status = run_detail.state
print(f"Current status: {status}")
```

6.6.3. Listing runs

Retrieve a list of runs with support for filtering, sorting, and pagination.

- To list all runs, use the **list_runs** method:

```
runs = client.list_runs()
```

- To list runs for a specific experiment:

```
runs = client.list_runs(experiment_id=experiment.experiment_id)
```

where:

experiment_id

Specifies the unique identifier of the experiment whose runs you want to list.

- To list runs with filtering, sorting, and pagination:

```
import json
run_filter = json.dumps({
    "predicates": [{
        "operation": "EQUALS",
        "key": "state",
        "stringValue": "RUNNING",
    }]
})
runs = client.list_runs(
    filter=run_filter,
    sort_by='created_at desc',
    page_size=50
)

for run in runs.runs:
    print(f"Run: {run.display_name} - Status: {run.state}")
```

where:

filter

Specifies a JSON-serialized filter to narrow results. The filter uses predicates with operation types such as **EQUALS**.

sort_by

Specifies the field and order for sorting results. Use **created_at desc** for newest first.

page_size

Specifies the maximum number of runs to return per page.

6.6.4. Managing run lifecycle

Control the lifecycle of runs by terminating, deleting, archiving, or unarchiving them.

- To cancel a running pipeline, use the **terminate_run** method:

```
client.terminate_run(run.run_id)
```

where:

run_id

Specifies the unique identifier of the run to terminate.

- To delete a run, use the **delete_run** method:

```
client.delete_run(run.run_id)
```

where:

run_id

Specifies the unique identifier of the run to delete.

- To archive a run, use the **archive_run** method:

```
client.archive_run(run.run_id)
```

where:

run_id

Specifies the unique identifier of the run to archive.

6.6.5. Additional resources

- [Overview of pipeline runs](#)
- [Working with experiments by using the SDK](#)
- [Kubeflow Pipelines SDK API Reference](#)

6.7. WORKFLOW EXAMPLES FOR THE KUBEFLOW PIPELINES SDK

Use these complete workflow examples to understand how to combine Kubeflow Pipelines SDK operations into end-to-end pipeline automation workflows.

These workflow examples require the Kubeflow Pipelines SDK to be installed and authenticated with the pipeline server. For more information about how to install the Kubeflow Pipelines SDK, see [Installing the Kubeflow Pipelines SDK](#).

6.7.1. End-to-end workflow example

The following example demonstrates a complete end-to-end workflow: creating an experiment, uploading a pipeline, submitting a run, and monitoring its execution.



NOTE

In the Red Hat OpenShift AI dashboard interface, experiments appear as **Run groups** in the **Runs** table. However, the REST API and SDK client use the term "experiment."

Set the **PIPELINE_HOST** environment variable to your pipeline server URL:

```
export PIPELINE_HOST=<your_pipeline_server_url>
```

where:

<your_pipeline_server_url>

Specifies the URL of your AI pipelines server, for example, **<https://ds-pipeline-dspa.apps.example.com>**.

```
#!/usr/bin/env python3
"""
Complete example of using AI Pipelines REST API
"""

import kfp
from kfp import Client
import time
import logging
import os

# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def main():
    # Initialize client
    pipeline_host = os.getenv('PIPELINE_HOST')
    if not pipeline_host:
        raise ValueError("PIPELINE_HOST environment variable not set")
    client = Client(host=pipeline_host)

    try:
        # 1. Create or get experiment
        experiment_name = 'rest-api-example'
        try:
            experiment = client.get_experiment(experiment_name=experiment_name)
```

```

        logger.info(f"Using existing experiment: {experiment.experiment_id}")
    except:
        experiment = client.create_experiment(
            name=experiment_name,
            description='Example experiment using REST API'
        )
        logger.info(f"Created new experiment: {experiment.experiment_id}")

# 2. Upload pipeline
# Replace '<path_to_pipeline_yaml>' with your compiled pipeline file path
pipeline = client.upload_pipeline(
    pipeline_package_path='<path_to_pipeline_yaml>',
    pipeline_name='example-pipeline',
    description='Example pipeline for REST API demo'
)
logger.info(f"Uploaded pipeline: {pipeline.pipeline_id}")

# 3. Submit run
run = client.run_pipeline(
    experiment_id=experiment.experiment_id,
    job_name=f'example-run-{int(time.time())}',
    pipeline_id=pipeline.pipeline_id,
    params={
        'input_data': 'gs://your-bucket/data.csv',
        'model_name': 'example-model',
        'epochs': 10
    }
)
logger.info(f"Submitted run: {run.run_id}")

# 4. Monitor run
logger.info("Monitoring run progress...")
client.wait_for_run_completion(run.run_id, timeout=3600)

# 5. Get final results
run_detail = client.get_run(run.run_id)
final_status = run_detail.state
logger.info(f"Run completed with status: {final_status}")

if final_status.lower() == 'succeeded':
    logger.info("Pipeline executed successfully!")
else:
    logger.error(f"Pipeline failed with status: {final_status}")

except Exception as e:
    logger.error(f"Error in pipeline execution: {str(e)}")
    raise

if __name__ == '__main__':
    main()

```

This example demonstrates the following actions:

- Initializing the SDK client
- Creating or retrieving an experiment

- Uploading a pipeline from a local YAML file
- Submitting a pipeline run with parameters
- Monitoring run completion
- Retrieving final run status

6.7.2. Example: Pagination for large result sets

Implement pagination for efficient retrieval of large datasets by iterating through pages by using page tokens.

The following example demonstrates pagination for retrieving all pipelines:

```
def get_all_pipelines(client):
    """Efficiently retrieve all pipelines using pagination."""
    all_pipelines = []
    page_token = None

    while True:
        response = client.list_pipelines(
            page_size=100,
            page_token=page_token
        )

        all_pipelines.extend(response.pipelines)

        if not response.next_page_token:
            break

        page_token = response.next_page_token

    return all_pipelines
```

where:

page_size

Specifies the number of items to retrieve per page. Use a reasonable page size such as 100.

page_token

Specifies the token for retrieving the next page of results.

6.7.3. Additional resources

- [Kubeflow Pipelines SDK Documentation](#)
- [Kubeflow Pipelines API Reference](#)

CHAPTER 7. WORK WITH PIPELINES IN JUPYTERLAB

7.1. JUPYTERLAB VISUAL PIPELINE EDITOR

You can use Elyra to create visual end-to-end pipeline workflows in JupyterLab. Elyra is an extension for JupyterLab that provides you with a Pipeline Editor to create pipeline workflows that can be executed in OpenShift AI.

You can access the Elyra extension within JupyterLab when you create the most recent version of one of the following workbench images:

- Standard Data Science
- PyTorch
- TensorFlow
- TrustyAI
- ROCm-PyTorch
- ROCm-TensorFlow

The Elyra pipeline editor is only available in specific workbench images. To use Elyra, the workbench must be based on a JupyterLab image. The Elyra extension is not available in code-server or RStudio IDEs. The workbench must also be derived from the Standard Data Science image. It is not available in Minimal Python or CUDA-based workbenches. All other supported JupyterLab-based workbench images have access to the Elyra extension.

When you use the Pipeline Editor to visually design your pipelines, minimal coding is required to create and run pipelines. For more information about Elyra, see [Elyra Documentation](#). For more information about the Pipeline Editor, see [Visual Pipeline Editor](#). After you have created your pipeline, you can run it locally in JupyterLab, or remotely using AI pipelines in OpenShift AI.

The pipeline creation process consists of the following tasks:

- Create a project that contains a workbench.
- Create a pipeline server.
- Create a new pipeline in the Pipeline Editor in JupyterLab.
- Develop your pipeline by adding Python notebooks or Python scripts and defining their runtime properties.
- Define execution dependencies.
- Run or export your pipeline.

Before you can run a pipeline in JupyterLab, your pipeline instance must contain a runtime configuration. A runtime configuration defines connectivity information for your pipeline instance and S3-compatible cloud storage.

If you create a workbench as part of a project, a default runtime configuration is created automatically. However, if you create a workbench from the **Start basic workbench** tile in the OpenShift AI dashboard, you must create a runtime configuration before you can run your pipeline in JupyterLab. For more

information about runtime configurations, see [Runtime Configuration](#). As a prerequisite, before you create a workbench, ensure that you have created and configured a pipeline server within the same project as your workbench.

You can use S3-compatible cloud storage to make data available to your notebooks and scripts while they are executed. Your cloud storage must be accessible from the machine in your deployment that runs JupyterLab and from the cluster that hosts AI pipelines. Before you create and run pipelines in JupyterLab, ensure that you have your s3-compatible storage credentials readily available.

Additional resources

- [Elyra Documentation](#)
- [Visual Pipeline Editor](#)
- [Runtime Configuration](#).

7.2. ACCESS THE PIPELINE EDITOR

You can use Elyra to create visual end-to-end pipeline workflows in JupyterLab. Elyra is an extension for JupyterLab that provides you with a Pipeline Editor to create pipeline workflows that can execute in OpenShift AI.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project.
- You have created and configured a pipeline server within the project that contains your workbench.



IMPORTANT

To ensure that the runtime configuration is created automatically, you must create the pipeline server before you create the workbench.

- You have created a workbench with a workbench image that contains the Elyra extension (Standard Data Science, TensorFlow, TrustyAI, ROCm-PyTorch, ROCm-TensorFlow, or PyTorch), as described in [Creating a workbench and selecting an IDE](#).
- You have started the workbench and opened the JupyterLab interface, as described in [Accessing your workbench IDE](#).



IMPORTANT

The Elyra pipeline editor is available in specific workbench images only. To use Elyra, the workbench must be based on a JupyterLab image. The Elyra extension is not available in code-server or RStudio IDEs. The workbench must also be derived from the Standard Data Science image. It is not available in Minimal Python or CUDA-based workbenches. All other supported JupyterLab-based workbench images have access to the Elyra extension.

- You have access to S3-compatible storage.

Procedure

1. After you open JupyterLab, confirm that the JupyterLab launcher is automatically displayed.
2. In the **Elyra** section of the JupyterLab launcher, click the **Pipeline Editor** tile.
The Pipeline Editor opens.

Verification

- You can view the Pipeline Editor in JupyterLab.

7.3. DISABLE NODE CACHING IN ELYRA

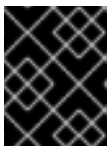
You can use the Elyra feature in OpenShift AI to cache components, or "nodes", within your AI pipelines. When a pipeline component runs, Elyra stores its outputs by default. In subsequent runs, if Elyra detects that a particular component has already been executed and its inputs have not changed, it reuses the cached outputs instead of re-running the entire component.

For more information about AI pipelines caching, see [Overview of pipelines caching](#).

If you do not want your component output to be cached, you can disable this feature for a single node or for all of the nodes in your pipeline.

Prerequisites

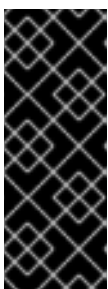
- You have logged in to Red Hat OpenShift AI.
- You have created a project.
- You have created and configured a pipeline server within the project that contains your workbench.



IMPORTANT

To ensure that the runtime configuration is created by default, you must create the pipeline server before you create the workbench.

- You have created a workbench with a workbench image that contains the Elyra extension (Standard Data Science, TensorFlow, TrustyAI, ROCm-PyTorch, ROCm-TensorFlow, or PyTorch), as described in [Creating a workbench and selecting an IDE](#).
- You have started the workbench and opened the JupyterLab interface, as described in [Accessing your workbench IDE](#).



IMPORTANT

The Elyra pipeline editor is available in specific workbench images only. To use Elyra, the workbench must be based on a JupyterLab image that includes the Elyra extension. The Elyra extension is not available in code-server or RStudio IDEs, and it is not included in Minimal Python workbenches. Supported JupyterLab-based images such as Standard Data Science, TensorFlow, PyTorch, TrustyAI, and ROCm variants include the Elyra extension.

- You have access to S3-compatible storage.

- You have created a pipeline in JupyterLab.

Procedure

1. After you open JupyterLab, confirm that the JupyterLab launcher is automatically displayed.
2. Open the pipeline that includes the nodes that you want to modify.
3. Right-click the node that you want to edit, and then click **Open Properties**.
4. To disable caching on a single node, complete the following steps:
 - a. Click the **Node Properties** tab in the slide-out menu on the right.
 - b. Under **Additional Properties**, click the option bar under **Disable node caching** that is automatically populated with **Use runtime default**
 - c. Select **True**.
5. To disable caching for all nodes on your pipeline, complete the following steps:
 - a. Click the **Pipeline Properties** tab in the slide-out menu on the right.
 - b. Under **Node Defaults**, click the option bar under **Disable node caching** that is automatically populated with **Use runtime default**
 - c. Select **True**.

Verification

- To verify that caching is disabled for a single node, check that node runs are re-executed in your target runtime environment.
- To verify that caching is disabled for all nodes on your pipeline, check that your entire pipeline runs are re-executed in your target runtime environment.

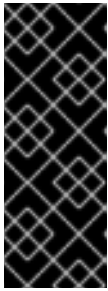
7.4. CREATE A RUNTIME CONFIGURATION

If you create a workbench as part of a project, a default runtime configuration is created automatically. However, if you create a workbench from the **Start basic workbench** tile in the OpenShift AI dashboard, you must create a runtime configuration before you can run your pipeline in JupyterLab. This enables you to specify connectivity information for your pipeline instance and S3-compatible cloud storage.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have access to S3-compatible cloud storage.
- You have created a project that contains a workbench.
- You have created and configured a pipeline server within the project that contains your workbench.

- You have created and launched a workbench from a workbench image that contains the Elyra extension (Standard Data Science, TensorFlow, TrustyAI, ROCm-PyTorch, ROCm-TensorFlow, or PyTorch).



IMPORTANT

The Elyra pipeline editor is available in specific workbench images only. To use Elyra, the workbench must be based on a JupyterLab image. The Elyra extension is not available in code-server or RStudio IDEs. The workbench must also be derived from the Standard Data Science image. It is not available in Minimal Python or CUDA-based workbenches. All other supported JupyterLab-based workbench images have access to the Elyra extension.

Procedure

1. In the left sidebar of JupyterLab, click **Runtimes** ().
2. Click the **Create new runtime configuration** button ().
The **Add new Pipelines runtime configuration** page opens.
3. Complete the relevant fields to define your runtime configuration.
 - a. In the **Display Name** field, enter a name for your runtime configuration.
 - b. Optional: In the **Description** field, enter a description to define your runtime configuration.
 - c. Optional: In the **Tags** field, click **Add Tag** to define a category for your pipeline instance.
Enter a name for the tag and press Enter.
 - d. Define the credentials of your pipeline:
 - i. In the **API endpoint** field, enter the API endpoint of your pipeline. Do not specify the pipelines namespace in this field.



IMPORTANT

With Elyra, you can now use a service-based URL instead of the route-based URL by including the port number. Using the service-based URL allows your pipeline to access the service directly.

- ii. In the **Public API endpoint** field, enter the public API endpoint of your pipeline.



IMPORTANT

You can obtain the pipeline API endpoint from the **Develop & train → Pipelines → Runs** page in the dashboard. Copy the relevant endpoint and enter it in the **Public API endpoint** field.

- iii. Optional: In the **User namespace** field, enter the relevant user namespace to run pipelines.
- iv. From the **Authentication Type** list, select the authentication type required to authenticate your pipeline.



IMPORTANT

If you created a workbench directly from the **Start basic workbench** tile on the dashboard, select **EXISTING_BEARER_TOKEN** from the **Authentication Type** list.

- v. In the **API endpoint username** field, enter the user name required for the authentication type.
- vi. In the **API endpoint password or token**, enter the password or token required for the authentication type.



IMPORTANT

To obtain the pipeline API endpoint token, in the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**. After you have logged in, click **Display token** and copy the value of **--token=** from the **Log in with this token** command.

- e. Define the connectivity information of your S3-compatible storage:
 - i. In the **Cloud Object Storage Endpoint** field, enter the endpoint of your S3-compatible storage. For more information about Amazon s3 endpoints, see [Amazon Simple Storage Service endpoints and quotas](#).
 - ii. Optional: In the **Public Cloud Object Storage Endpoint** field, enter the URL of your S3-compatible storage.
 - iii. In the **Cloud Object Storage Bucket Name** field, enter the name of the bucket where your pipeline artifacts are stored. If the bucket name does not exist, it is created automatically.
 - iv. From the **Cloud Object Storage Authentication Type** list, select the authentication type required to access to your S3-compatible cloud storage. If you use AWS S3 buckets, select **KUBERNETES_SECRET** from the list.
 - v. In the **Cloud Object Storage Credentials Secret** field, enter the secret that contains the storage user name and password. This secret is defined in the relevant user namespace, if applicable. In addition, it must be stored on the cluster that hosts your pipeline runtime.
 - vi. In the **Cloud Object Storage Username** field, enter the user name to connect to your S3-compatible cloud storage, if applicable. If you use AWS S3 buckets, enter your AWS Secret Access Key ID.
 - vii. In the **Cloud Object Storage Password** field, enter the password to connect to your S3-compatible cloud storage, if applicable. If you use AWS S3 buckets, enter your AWS Secret Access Key.
- f. Click **Save & Close**.

Verification

- The runtime configuration that you created is displayed on the **Runtimes** tab () in the left sidebar of JupyterLab.

7.5. UPDATE A RUNTIME CONFIGURATION

To ensure that your runtime configuration is accurate and updated, you can change the settings of an existing runtime configuration.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have access to S3-compatible storage.
- You have created a project that contains a workbench.
- You have created and configured a pipeline server within the project that contains your workbench.
- A previously created runtime configuration is available in the JupyterLab interface.
- You have created and launched a workbench from a workbench image that contains the Elyra extension (Standard Data Science, TensorFlow, TrustyAI, ROCm-PyTorch, ROCm-TensorFlow, or PyTorch).



IMPORTANT

The Elyra pipeline editor is available in specific workbench images only. To use Elyra, the workbench must be based on a JupyterLab image. The Elyra extension is not available in code-server or RStudio IDEs. The workbench must also be derived from the Standard Data Science image. It is not available in Minimal Python or CUDA-based workbenches. All other supported JupyterLab-based workbench images have access to the Elyra extension.

Procedure

1. In the left sidebar of JupyterLab, click **Runtimes** ().
2. Hover the cursor over the runtime configuration that you want to update and click the **Edit** button ().
The **Pipelines runtime configuration** page opens.
3. Fill in the relevant fields to update your runtime configuration.
 - a. In the **Display Name** field, update name for your runtime configuration, if applicable.
 - b. Optional: In the **Description** field, update the description of your runtime configuration, if applicable.
 - c. Optional: In the **Tags** field, click **Add Tag** to define a category for your pipeline instance. Enter a name for the tag and press Enter.

d. Define the credentials of your pipeline:

- i. In the **API Endpoint** field, update the API endpoint of your AI pipeline, if applicable. Do not specify the pipelines namespace in this field.



IMPORTANT

With Elyra, you can now use a service-based URL instead of the route-based URL by including the port number. Using the service-based URL allows your AI pipeline to access the service directly.

- ii. In the **Public API endpoint** field, update the API endpoint of your AI pipeline, if applicable.
- iii. Optional: In the **User namespace** field, update the relevant user namespace to run pipelines, if applicable.
- iv. From the **Authentication Type** list, select a new authentication type required to authenticate your pipeline, if applicable.



IMPORTANT

If you created a workbench directly from the **Start basic workbench** tile on the dashboard, select **EXISTING_BEARER_TOKEN** from the **Authentication Type** list.

- v. In the **API endpoint username** field, update the user name required for the authentication type, if applicable.
- vi. In the **API endpoint password or token**, update the password or token required for the authentication type, if applicable.



IMPORTANT

To obtain the pipelines API endpoint token, in the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**. After you have logged in, click **Display token** and copy the value of **--token=** from the **Log in with this token** command.

e. Define the connectivity information of your S3-compatible storage:

- i. In the **Cloud Object Storage Endpoint** field, update the endpoint of your S3-compatible storage, if applicable. For more information about Amazon s3 endpoints, see [Amazon Simple Storage Service endpoints and quotas](#) .
- ii. Optional: In the **Public Cloud Object Storage Endpoint** field, update the URL of your S3-compatible storage, if applicable.
- iii. In the **Cloud Object Storage Bucket Name** field, update the name of the bucket where your pipeline artifacts are stored, if applicable. If the bucket name does not exist, it is created automatically.

- iv. From the **Cloud Object Storage Authentication Typelist**, update the authentication type required to access to your S3-compatible cloud storage, if applicable. If you use AWS S3 buckets, you must select **USER_CREDENTIALS** from the list.
 - v. Optional: In the **Cloud Object Storage Credentials Secret** field, update the secret that contains the storage user name and password, if applicable. This secret is defined in the relevant user namespace. You must save the secret on the cluster that hosts your pipeline runtime.
 - vi. Optional: In the **Cloud Object Storage Username** field, update the user name to connect to your S3-compatible cloud storage, if applicable. If you use AWS S3 buckets, update your AWS Secret Access Key ID.
 - vii. Optional: In the **Cloud Object Storage Password** field, update the password to connect to your S3-compatible cloud storage, if applicable. If you use AWS S3 buckets, update your AWS Secret Access Key.
- f. Click **Save & Close**.

Verification

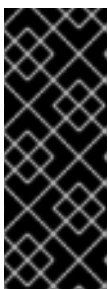
- The runtime configuration that you updated is shown on the **Runtimes** tab () in the left sidebar of JupyterLab.

7.6. DELETE A RUNTIME CONFIGURATION

After you have finished using your runtime configuration, you can delete it from the JupyterLab interface. After deleting a runtime configuration, you cannot run pipelines in JupyterLab until you create another runtime configuration.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project that contains a workbench.
- You have created and configured a pipeline server within the project that contains your workbench.
- A previously created runtime configuration is visible in the JupyterLab interface.
- You have created and launched a workbench from a workbench image that contains the Elyra extension (Standard Data Science, TensorFlow, TrustyAI, ROCm-PyTorch, ROCm-TensorFlow, or PyTorch).



IMPORTANT

The Elyra pipeline editor is available in specific workbench images only. To use Elyra, the workbench must be based on a JupyterLab image. The Elyra extension is not available in code-server or RStudio IDEs. The workbench must also be derived from the Standard Data Science image. It is not available in Minimal Python or CUDA-based workbenches. All other supported JupyterLab-based workbench images have access to the Elyra extension.

Procedure

1. In the left sidebar of JupyterLab, click **Runtimes** ().
2. Hover the cursor over the runtime configuration that you want to delete and click the **Delete Item** button ().
A dialog box opens prompting you to confirm the deletion of your runtime configuration.
3. Click **OK**.

Verification

- The runtime configuration that you deleted is no longer shown on the **Runtimes** tab () in the left sidebar of JupyterLab.

7.7. DUPLICATE A RUNTIME CONFIGURATION

To prevent you from re-creating runtime configurations with similar values in their entirety, you can duplicate an existing runtime configuration in the JupyterLab interface.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project that contains a workbench.
- You have created and configured a pipeline server within the project that contains your workbench.
- A previously created runtime configuration is visible in the JupyterLab interface.
- You have created and launched a workbench from a workbench image that contains the Elyra extension (Standard Data Science, TensorFlow, TrustyAI, ROCm-PyTorch, ROCm-TensorFlow, or PyTorch).



IMPORTANT

The Elyra pipeline editor is available in specific workbench images only. To use Elyra, the workbench must be based on a JupyterLab image. The Elyra extension is not available in code-server or RStudio IDEs. The workbench must also be derived from the Standard Data Science image. It is not available in Minimal Python or CUDA-based workbenches. All other supported JupyterLab-based workbench images have access to the Elyra extension.

Procedure

1. In the left sidebar of JupyterLab, click **Runtimes** ().
2. Hover the cursor over the runtime configuration that you want to duplicate and click the **Duplicate** button ().

Verification

- The runtime configuration that you duplicated is shown on the **Runtimes** tab () in the left sidebar of JupyterLab.

7.8. RUN A PIPELINE IN JUPYTERLAB

You can run pipelines that you have created in JupyterLab from the Pipeline Editor user interface. Before you can run a pipeline, you must create a project and a pipeline server. After you create a pipeline server, you must create a workbench within the same project as your pipeline server. Your pipeline instance in JupyterLab must contain a runtime configuration. If you create a workbench as part of a project, a default runtime configuration is created automatically. However, if you create a workbench from the **Start basic workbench** tile in the OpenShift AI dashboard, you must create a runtime configuration before you can run your pipeline in JupyterLab. A runtime configuration defines connectivity information for your pipeline instance and S3-compatible cloud storage.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have access to S3-compatible storage.
- You have created a pipeline in JupyterLab.
- You have opened your pipeline in the Pipeline Editor in JupyterLab.
- Your pipeline instance contains a runtime configuration.
- You have created and configured a pipeline server within the project that contains your workbench.
- You have created and launched a workbench from a workbench image that contains the Elyra extension (Standard Data Science, TensorFlow, TrustyAI, ROCm-PyTorch, ROCm-TensorFlow, or PyTorch).

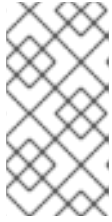


IMPORTANT

The Elyra pipeline editor is available in specific workbench images only. To use Elyra, the workbench must be based on a JupyterLab image. The Elyra extension is not available in code-server or RStudio IDEs. The workbench must also be derived from the Standard Data Science image. It is not available in Minimal Python or CUDA-based workbenches. All other supported JupyterLab-based workbench images have access to the Elyra extension.

Procedure

1. In the Pipeline Editor user interface, click **Run Pipeline** ().
The **Run Pipeline** dialog opens. The **Pipeline Name** field is automatically populated with the pipeline file name.



NOTE

After you run your pipeline, a pipeline experiment containing your pipeline run is automatically created on the **Develop & train → Experiments** page in the OpenShift AI dashboard. The experiment name matches the name that you assigned to the pipeline.

2. Define the settings for your pipeline run.
 - a. From the **Runtime Configuration** list, select the relevant runtime configuration to run your pipeline.
 - b. Optional: Configure your pipeline parameters, if applicable. If your pipeline contains nodes that reference pipeline parameters, you can change the default parameter values. If a parameter is required and has no default value, you must enter a value.
3. Click **OK**.

Verification

- You can view the details of your pipeline run on the **Develop & train → Experiments** page in the OpenShift AI dashboard.
- You can view the output artifacts of your pipeline run. The artifacts are stored in your designated object storage bucket.

7.9. EXPORT A PIPELINE IN JUPYTERLAB

You can export pipelines that you have created in JupyterLab. When you export a pipeline, the pipeline is prepared for later execution, but is not uploaded or executed immediately. During the export process, any package dependencies are uploaded to S3-compatible storage. Also, pipeline code is generated for the target runtime.

Before you can export a pipeline, you must create a project and a pipeline server. After you create a pipeline server, you must create a workbench within the same project as your pipeline server. In addition, your pipeline instance in JupyterLab must contain a runtime configuration. If you create a workbench as part of a project, a default runtime configuration is created automatically. However, if you create a workbench from the **Start basic workbench** tile in the OpenShift AI dashboard, you must create a runtime configuration before you can export your pipeline in JupyterLab. A runtime configuration defines connectivity information for your pipeline instance and S3-compatible cloud storage.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project that contains a workbench.
- You have created and configured a pipeline server within the project that contains your workbench.
- You have access to S3-compatible storage.
- You have created a pipeline in JupyterLab.
- You have opened your pipeline in the Pipeline Editor in JupyterLab.

- Your pipeline instance contains a runtime configuration.
- You have created and launched a workbench from a workbench image that contains the Elyra extension (Standard Data Science, TensorFlow, TrustyAI, ROCm-PyTorch, ROCm-TensorFlow, or PyTorch).



IMPORTANT

The Elyra pipeline editor is available in specific workbench images only. To use Elyra, the workbench must be based on a JupyterLab image. The Elyra extension is not available in code-server or RStudio IDEs. The workbench must also be derived from the Standard Data Science image. It is not available in Minimal Python or CUDA-based workbenches. All other supported JupyterLab-based workbench images have access to the Elyra extension.

Procedure

1. In the Pipeline Editor user interface, click **Export Pipeline** ().
The **Export Pipeline** dialog opens. The **Pipeline Name** field is automatically populated with the pipeline file name.
2. Define the settings to export your pipeline.
 - a. From the **Runtime Configuration** list, select the relevant runtime configuration to export your pipeline.
 - b. From the **Export Pipeline as** select an appropriate file format
 - c. In the **Export Filename** field, enter a file name for the exported pipeline.
 - d. Select the **Replace if file already exists** check box to replace an existing file of the same name as the pipeline you are exporting.
 - e. Optional: Configure your pipeline parameters, if applicable. If your pipeline contains nodes that reference pipeline parameters, you can change the default parameter values. If a parameter is required and has no default value, you must enter a value.
3. Click **OK**.

Verification

- You can view the file containing the pipeline that you exported in your designated object storage bucket.

CHAPTER 8. DSPA COMPONENT TROUBLESHOOTING

This table displays common errors found in **DataSciencePipelinesApplication** (DSPA) components, along with the associated status, message, and proposed solution. The **Ready** condition type accumulates errors from various DSPA components, providing a status view of the DSPA deployment.

Type	Status	Error message and solution
ObjectStorageAvailable Ready	False False	Error message: Could not connect to Object Store: tls: failed to verify certificate: x509: certificate signed by unknown authority Solution: This issue occurs in clusters that use self-signed certificates with OpenShift AI version 2.9 or later. The AI pipelines manager cannot connect to the object storage because it does not trust the object storage SSL certificate. Therefore, the pipeline server cannot be created. Contact your IT operations administrator to add the relevant Certificate Authority bundle. For more information, see Working with certificates.
ObjectStorageAvailable Ready	False False	Error message: Could not connect to Object Store Deployment for component "ds-pipeline-pipelines-definition" is missing - prerequisite component might not yet be available. Deployment for component "ds-pipeline-persistenceagent-pipelines-definition" is missing - prerequisite component might not yet be available. Deployment for component "ds-pipeline-scheduledworkflow-pipelines-definition" is missing - prerequisite component might not yet be available. Solution: The AI pipelines manager might fail to connect to the object storage, and the pipeline server might not be created. Ensure that your object store credentials and connection information are accurate, and verify that the object store is accessible from within the project's associated OpenShift namespace. One common issue is that the object storage SSL certificate is not trusted, particularly if self-signed certificates are used. Verify and update your object storage credentials, then retry the operation.
ObjectStorageAvailable Ready	False False	Error message: Wrong credentials for Object Storage: Could not connect to (minio-my-project.apps.my-cluster.com), Error: The request signature we calculated does not match the signature you provided. Check your key and signing method. Solution: Provide the correct credentials for your object storage and retry the operation.

Type	Status	Error message and solution
DatabaseAvailable Ready	False False	Error message: FailingToDeploy: Dial tcp XXX.XX.XXX.XXX:3306 : i/o timeout Solution: If the issue persists beyond startup, check for network issues or misconfigurations in the database connection settings.
DatabaseAvailable Ready	False False	Error message: Unable to connect to external database: tls: failed to verify certificate: x509: certificate signed by unknown authority Solution: This issue can occur when you use any external database, such as Amazon RDS. The AI pipelines manager cannot connect to the database because it does not trust the database SSL certificate, preventing the creation of the pipeline server. Contact your IT operations administrator to add the relevant certificates. For more information, see Working with certificates
DatabaseAvailable Ready	False False	Error message: Error 1129: Host 'A.B.C.D' is blocked because of many connection errors. Solution: This issue might occur when using an external database, such as Amazon RDS. Initially, the pipeline server is created successfully. However, after some time, the OpenShift AI dashboard displays an "Error displaying pipelines" message, and the DSPA conditions indicate that the host is blocked due to multiple connection errors. For more information on how resolve this issue for an external Amazon RDS database, see Resolving "Host is blocked because of many connection errors" error in Amazon RDS for MySQL. Note: Clicking this link opens an external website.
APIServerReady Ready	False False	Error message: Route creation failed due to lengthy project name: Route.route.openshift.io is invalid: spec.host exceeds 63 characters. Solution: Ensure that the project name in OpenShift is less than 40 characters.
APIServerReady Ready	False False	Error message: FailingToDeploy: Component replica failed to create. Message: serviceaccount "ds-pipeline-sample" not found. Solution: If the failure persists for more than 25 seconds during DSPA startup, recreate the missing service account.

Type	Status	Error message and solution
PersistenceAgentReady Ready	False False	Error message: FailingToDeploy: Component's replica failed to create. Message: serviceaccount "ds-pipeline-persistenceagent-sample" not found. Solution: If the failure persists for more than 25 seconds during DSPA startup, recreate the missing service account.
ScheduledWorkflowReady Ready	False False	Error message: FailingToDeploy: Component's replica failed to create. Message: serviceaccount "ds-pipeline-scheduledworkflow-sample" not found. Solution: If the failure persists for more than 25 seconds during DSPA startup, recreate the missing service account.
MLMDProxyReady Ready	False False	Error message: Deploying: Component [ds-pipeline-scheduledworkflow-sample] is still deploying. Solution: Wait for DSPA startup to complete. If deployment fails after 25 seconds, check the logs for further information.

8.1. COMMON ERRORS ACROSS DSPA COMPONENTS

The following table lists errors that might occur across multiple DSPA components:

Deployment condition and condition type	Status	Error message and solution
Condition: Component Deployment Not Found Condition type: ComponentDeploymentNotFound	False	Error message: Deployment for component <component> is missing - prerequisite component might not yet be available. Solution: The deployment for the component does not exist. Typically, this issue occurs due to missing deployments or issues that occurred during creation.
Condition: Deployment Scaled Down Condition type: MinimumReplicasAvailable	False	Error message: Deployment for component <component> is scaled down. Solution: The component is unavailable as the deployment replica count is set to zero.
Condition: Component Failing to Progress Condition type: FailingToDeploy	False	Error message: Component <component> has failed to progress. Reason: <progressingCond.Reason>. Message: <progressingCond.Message> Solution: The deployment has stalled due to ProgressDeadlineExceeded or ReplicaSetCreateError issues, or similar.

Deployment condition and condition type	Status	Error message and solution
Condition: Replica Creation Failure Condition type: FailingToDeploy	False	Error message: Component's replica <component> has failed to create. Reason: <replicaFailureCond.Reason>. Message: <replicaFailureCond.Message> Solution: Replica creation has failed, typically due to an error in the replica set or with the service accounts.
Condition: Pod-Level Failures Condition type: FailingToDeploy	False	Error message: Concatenated failure messages for each pod. Solution: Deployment pods are in a failed state. Check the pod logs for further information.
Condition: Pod in CrashLoopBackOff Condition type: FailingToDeploy	False	Error message: Component <component> is in CrashLoopBackOff. Message from pod: <crashLoopBackOffMessage> Solution: Pod containers are failing repeatedly, often due to incorrect environment variables or missing service accounts.
Condition: Component Deploying (No Errors) Condition type: Deploying	False	Error message: Component <component> is deploying. Solution: The component deployment process is ongoing with no errors detected.
Condition: Component Minimally Available Condition type: MinimumReplicasAvailable	True	Error message: Component <component> is minimally available. Solution: The component is available, but with only the minimum number of replicas running.

CHAPTER 9. ADDITIONAL RESOURCES

- [Kubeflow Pipelines 2.0 Documentation](#)